

Observations on the BAYESIANKEYSEARCH with Applications to SIMON and SIMECK

Zezhou Hou¹, Zhenzhen Bao^{2,3,4(✉)}, Jinyu Lu² and Shaozhen Chen¹

¹ Information Engineering University, Zhengzhou, 450001, China
zezhouhou_hzz@163.com, chenshaozhen@vip.sina.com

² Institute for Network Sciences and Cyberspace, Tsinghua University, Beijing, 100084, China
zzbao@tsinghua.edu.cn, jinyu_smile@foxmail.com

³ Zhongguancun Laboratory, Beijing, China

⁴ State Key Laboratory of Cryptography and Digital Economy Security, Tsinghua University, Beijing, 100084, China

Abstract. In CRYPTO 2019, Gohr pioneered the integration of machine learning with differential cryptanalysis, demonstrating that differential-neural distinguishers can outperform classical techniques in distinguishing attacks. He also introduced a novel key-recovery strategy based on Bayesian optimization, termed BAYESIANKEYSEARCH, enhancing key recovery for SPECK32/64. However, the impact of parameter selection on the complexity and success probability of key-recovery attack using BAYESIANKEYSEARCH remains underexplored.

This paper investigates the impact of parameter selections on key-recovery effectiveness. Gohr’s key-recovery attack involves two stages, each using a cutoff value to filter candidate guesses for the last subkey and second-to-last subkey. Previous works selected these cutoffs independently. We propose connecting these cutoff selections, enhancing coordination between stages and improving the attack’s complexity and success probability.

Applying our parameter optimization, we enhance the single-key recovery attacks on 16-round SIMON32/64, 16-round and 17-round SIMECK32/64, achieving higher success rates and lower time complexities compared to previous works. Additionally, for related-key differential-neural attacks on SIMON32/64, we exploit both single-key and related-key features from cross-paired ciphertexts, developing advanced neural-distinguishers for up to 13 rounds. Using these neural-distinguishers combined with carefully selected classical differentials, we devise an 18-round related-key recovery attack on SIMON32/64. Our results validate the practical effectiveness of the proposed strategies and are expected to contribute to the advancement of machine learning-aided cryptanalysis.

Keywords: Neural Network · BAYESIANKEYSEARCH · Parameter Optimization · Key-recovery · SIMON32/64 · SIMECK32/64

1 Introduction

Machine learning (ML), a pivotal branch of artificial intelligence, has profoundly impacted numerous domains including computer vision and machine reasoning. Its core functionalities – including pattern recognition, classification, and decision-making – have intrinsic parallels with cryptographic tasks, especially in the realm of cryptanalysis.

The intersection between cryptography and machine learning has been highlighted by Rivest in 1991 [Riv93]. In recent years, advances in hardware and theoretical frameworks have propelled the application of machine learning techniques to cryptographic tasks,

particularly in side-channel analysis[PSK⁺18, KPH⁺19, PPM⁺23], demonstrating its potential for cryptography.

A landmark development in the integration of machine learning with the classical cryptanalysis of the symmetric-key ciphers emerged at CRYPTO 2019 through Gohr’s pioneering work [Goh19]. Gohr transformed the task of distinguishing output pairs of a block cipher – generated from inputs with a given difference – from random data pair into a supervised learning problem, thereby utilizing neural networks to enhance the accuracy of cryptographic distinguishers. Quite surprisingly, when applying his framework to round-reduced SPECK32/64 [BSS⁺13], the trained neural network, known as neural distinguisher ($\mathcal{ND}$), obtains the non-negligible accuracy in classifying random from real ciphertext pairs.

The work in [Goh19] extends beyond mere distinguishing attacks. Gohr introduced BAYESIANKEYSEARCH, a highly selective key search policy employing Bayesian optimization and $\mathcal{ND}$ s, which significantly reduces the complexity of key guessing by narrowing down the pool of potential candidates. Furthermore, by integrating a classical differential ($\mathcal{CD}$) with $\mathcal{ND}$ s to form hybrid distinguishers, Gohr implemented an optimized key-recovery attack that demonstrates significantly reduced complexity relative to the traditional cryptanalysis.

Inspired by Gohr’s work, researchers are increasingly exploring ML-aided cryptanalysis. In EUROCRYPT 2021, Benamira *et al.* [BGPT21] analyzed the internal workings of Gohr’s $\mathcal{ND}$ s, demonstrating that the $\mathcal{ND}$ for SPECK32/64 primarily leverages the differential distribution of ciphertext pairs, as well as differential distributions in the penultimate and antepenultimate rounds. Bao *et al.* [BLYZ23] further concluded that using ciphertext pairs as training samples allows $\mathcal{ND}$ s to exploit the differential distribution conditioned on partially known inputs to the last non-linear operation of SPECK. Beyond the understanding the internal mechanisms of $\mathcal{ND}$, efforts have been made to enhance $\mathcal{ND}$ s performance by modifying training sample formats [CSYY23, LLS⁺24], optimizing hyperparameters [GLN22], and utilizing more complex training methods and neural networks[ZWW22, BGHR23]. For a more comprehensive review of related works in neural differential cryptanalysis after Gohr’s seminal work, please refer to [GHHP24].

In the context of neural distinguisher-based key-recovery attacks, Lyu *et al.* [LTZ22] described their empirical observations. They presented empirical evidence suggesting that the choice of the parameters influenced both the success rate and the time complexity of the attack. However, they reported no clear methodology to optimally adjust these parameters. In addition, Bao *et al.* [BGL⁺22] offered some rules of thumb for selecting parameters in the neural differential attack, particularly regarding the cutoffs used to filter the ciphertext pairs.¹ In their method, the cutoff for the last subkey is decided based on score distributions from both right and wrong ciphertext structures. For the second-to-last subkey, since the score distributions are more clearly separated, a cutoff is selected with less consideration – large enough to block bad guesses but still allowing almost correct ones. Notably, the two cutoffs are determined independently.

However, since the second cutoff directly determines when the entire BAYESIANKEYSEARCH stops, it is critical for the attack’s success probability. Beyond score distributions, the number of wrong structures and key candidates passing the first cutoff should be considered when setting the second. Thus, the second cutoff should be decided alongside the first, as the first affects how many wrong candidates reach the second. From another perspective, the two BAYESIANKEYSEARCH stages are linked by the subkeys candidates recommended from the first, so the parameters for the latter should be decided with consideration of the first. With the defects of previous works, there are several questions to explore. That might include the following:

¹The comparison with our work is shown in Appendix D.

- Given the fixed parameters of both BAYESIANKEYSEARCH, how to choose the values of two cutoffs to achieve the full potential of $\mathcal{ND}$ s?
- Further, for the case that performing a key recovery is time-consuming, given multiple groups of parameters of both BAYESIANKEYSEARCH, how to choose the best parameters from these parameter candidates without actually performing the key-recovery attack?

Our contribution. This work advances machine learning-aided cryptanalysis by developing an optimized parameter and cutoff selection strategy for BAYESIANKEYSEARCH to reduce its time complexity. Applying the parameter optimization strategy, we present the refined key recovery attacks on 16-round SIMON32/64, 16-round and 17-round SIMECK32/64 in single-key settings that achieves significantly improved success probabilities while reducing time complexity compared to existing attacks. Additionally, we leverage both single-key and related-key features derived from cross-paired ciphertexts in related-key neural-differential attacks. We again demonstrate the effectiveness of these strategies through practical key-recovery attacks on round-reduced SIMON32/64 in related-key settings, showcasing enhanced success rates and reduced computational overhead. The concrete contributions include the following.²

- **Optimizing parameters and cutoffs for BAYESIANKEYSEARCH.**
 - For the first BAYESIANKEYSEARCH, the cutoff is designed to filter out as many wrong ciphertext structures (and bad last subkey guesses) as possible while allowing right ciphertext structures (with good last subkey guesses) to pass. We analyze the relationship between the upper bound of this cutoff and the success probability, and propose a method for tuning the cutoff using a set of precomputed data.
 - In the second BAYESIANKEYSEARCH, the cutoff is designed to prevent any wrong structures (and bad second-to-last subkey guesses) from passing. Given the fixed parameters of this stage, the probability of wrong structures passing the second cutoff primarily depends on the number of wrong structures that escaped the first cutoff and the value of the second cutoff. Consequently, we derive a lower bound for the second cutoff by considering the number of candidates passing the first cutoff.
 - By treating both stages as a unified system and considering the influence of both cutoffs on the probability of the right structure passing through, we propose a coordinated strategy for selecting parameters and cutoffs for key-recovery attacks employing two BAYESIANKEYSEARCH stages. Given multiple parameter sets for the two BAYESIANKEYSEARCHs, the algorithm selects the optimal parameter sets based on their corresponding success probabilities.
- **Practical key recovery attacks on round-reduced SIMON32/64.**

Applying our parameter optimization, we develop improved 16-round neural-differential-based key-recovery attacks on SIMON32/64. These attacks show advantages in both time complexity and success rate compared to previous $\mathcal{ND}$ -based attacks. Furthermore, we design a new training sample format for related-key neural distinguishers ($\mathcal{RK}\text{-}\mathcal{ND}$ s), which improves the accuracy without using more plaintexts. Using this format, we train enhanced 11-, 12-, and 13-round distinguishers for SIMON32/64. Leveraging the 12- and 13-round $\mathcal{RK}\text{-}\mathcal{ND}$ s along with multiple 3-round characteristics, with the cutoffs recommended by our choice strategy, we devise a practical 18-round related-key recovery attack. The results, shown in Table 1, demonstrate

²Please refer to our [source code repository](#) for the implementation details.

that the theoretical success rate closely matches experimental results, validating the effectiveness of the parameters and cutoff choice algorithm.

- **Practical key recovery attacks on round-reduced SIMECK32/64.**

Applying our parameter optimization, we develop improved 16-round and 17-round neural-differential-based key-recovery attacks on SIMECK32/64. These attacks show advantages in both time complexity and success rate compared to previous $\mathcal{ND}$ -based attacks. This further validates the adaptability of our parameter optimization strategy across the neural-differential attacks.

Table 1: Summary of key-recovery attacks on SIMON32/64 and SIMECK32/64

Target Cipher	Rounds	Distinguisher	Configuration	Time	Data	Success Rate	Key Space	Ref.
SIMON32/64	16	$\mathcal{ND}$	1+3+11+1	$2^{44.21*}$	2^{21}	49.5%	2^{64}	[BGL ⁺ 22]
	16	$\mathcal{ND}$	1+3+11+1	$2^{43.81*,\dagger}$	2^{21}	67.7%	2^{64}	Section 6.1
	17	$\mathcal{ND}$	1+4+11+1	$2^{45.56}$	2^{28}	9%	2^{64}	[ZWW22]
	18	$\mathcal{DD}$	1+13+4	2^{46}	$2^{31.2}$	63%	2^{64}	[ALLW15]
	18	$\mathcal{IDD}$	5+11+2	$2^{61.14}$	2^{32}	-	2^{64}	[WLW ⁺ 14]
	18	$\mathcal{RK-ND}$	1+3+13+1	$2^{43.08*\ddagger}$	2^{29*}	65.9%	2^{63}	Section 6.2
	21	$\mathcal{DD}$	4+13+4	$2^{55.25}$	$2^{31.0}$	51%	2^{64}	[WWJZ18]
	22	$\mathcal{DD}$	3+14+5	$2^{58.76}$	$2^{32.0}$	31.5%	2^{64}	[QHS16]
	23	$\mathcal{LD}$	5+13+5	$2^{59.59}$	$2^{31.19}$	57.1%	2^{64}	[ZLQ ⁺ 21]
	23	$\mathcal{LD}$	5+14+4	2^{50}	$2^{30.59}$	-	2^{64}	[AAA ⁺ 15]
	23	$\mathcal{D-MITM}$	1+4+14+4	$2^{63.52}$	2^{32}	-	2^{64}	[DZW25]
	24	$\mathcal{LD}$	5+14+5	$2^{63.9}$	$2^{30.59}$	-	2^{64}	[AAA ⁺ 15]
	24	$\mathcal{D-MITM}$	1+4+14+5	$2^{63.8}$	2^{32}	-	2^{64}	[DZW25]
SIMECK32/64	15	$\mathcal{LD}$	-	2^{56}	2^{32}	86.7%	2^{64}	[LTZ22]
	15	$\mathcal{ND}$	1+3+10+1	$2^{38.90\nabla}$	2^{24}	88%	2^{64}	[LTZ22]
	16	$\mathcal{ND}$	1+3+11+1	$2^{38.189}$	2^{24*}	100%	2^{64}	[ZLWL23]
	16	$\mathcal{ND}$	1+3+11+1	$2^{37.29}$	2^{24*}	100%	2^{64}	Section 7.3
	17	$\mathcal{ND}$	1+3+12+1	$2^{45.037}$	2^{26*}	30%	2^{64}	[ZLWL23]
	17	$\mathcal{ND}$	1+3+12+1	$2^{43.58}$	2^{26*}	34%	2^{64}	Section 7.4
	18	$\mathcal{LD}$	3+11+4	$2^{63.5}$	2^{31}	47.7%	2^{64}	[Bag15]
	19	$\mathcal{DD}$	2+13+4	2^{40}	2^{31}	-	2^{64}	[KR15]
	22	$\mathcal{DD}$	4+13+5	$2^{57.9}$	2^{32}	47.1%	2^{64}	[QHS16]
	22	$\mathcal{DD}$	4+14+4	$2^{56}E + 2^{54.3}A^{\S}$	2^{32}	73%	2^{64}	[HWZ18]
	23	$\mathcal{LD}$	5+13+5	$2^{58.3}$	$2^{31.5}$	7%	2^{64}	[LPS21]

1. $\mathcal{DD}$: differential distinguisher; $\mathcal{IDD}$: impossible differential distinguisher; $\mathcal{LD}$: linear distinguisher; $\mathcal{ND}$: neural-differential distinguisher; $\mathcal{RK-ND}$: neural-differential distinguisher in the related-key setting; $\mathcal{D-MITM}$: differential meet-in-the-middle cryptanalysis;

2. *: The complexity is calculated in the worst case.

3. $\dagger$: The time complexity in [BGL⁺22] is given under the assumption that one second equals the time of 2^{28} executions of SIMON32/64. And the time of the improved key-recovery attack equals the time of $2^{-0.40}$ executions of the attack in [BGL⁺22]. Thus, the time complexity in the single-key scenario is $2^{43.81}$.

4. $\ddagger$: The attack time in the related-key scenario equals $2^{43.08}$ 18-round encryption operations.

5. ∇ : The value is obtained from the comparison table in [ZLWL23].

6. $\S$: 'A' represents a XOR addition operation and 'E' represents the encryption operation of attacked rounds.

7. In the column entitled 'Configuration', the $1+s+r+1$ refers to that the neural-differential attack employs an r -round main and an $(r-1)$ -round helper $\mathcal{ND}$; a short s -round differential. The first '1' refers to the '1-round free' and the last '1' refers to the '1-round key-guessing'.

Organisation of the paper: The rest of the paper is organized as follows. Section 2 provides preliminaries on machine learning-aided cryptanalysis. Section 3 introduces a new sample format for related-key neural distinguishers. In Section 4, we describe the differential characteristics in the single-key and related-key settings. Section 5 presents the strategy for selecting cutoffs and parameters for BAYESIANKEYSEARCH-based key-recovery attacks. The 16-round key-recovery attack in the single-key setting and the 18-round attack in the related-key setting on SIMON32/64 are presented in Section 6. The improved 16-round and 17-round key-recovery attacks on SIMECK32/64 are presented in Section 7.

The complexity of the parameter selection strategy is presented in Section 8. Section 9 concludes the paper and suggests areas for future work.

2 Preliminary

2.1 Notations

Denote by (L_i, R_i) the left and right branches of a state after the encryption of i rounds. Denote by $L_i || R_i$ the concatenation of L_i and R_i . Denote by $L_i[j]$ (resp. $R_i[j]$) the j -th bit of L_i (resp. R_i) counted starting from 0, where $L_i[0]$ (resp. $R_i[0]$) is the LSB (Least Significant Bit) of L_i (resp. R_i), *i.e.*, $L_i = L_i[n-1] || L_i[n-2] || \dots || L_i[0]$ and $R_i = R_i[n-1] || R_i[n-2] || \dots || R_i[0]$. Denote by $L_i[i_1, i_2, \dots, i_t]$ the set of i_1 -th bit, ..., i_t -th bit of L_i .

2.2 Overview of Differential-Neural Cryptanalysis

In CRYPTO 2019, Gohr [Goh19] proposed the differential-neural cryptanalysis, whose work mainly consisted of utilizing neural networks as the distinguishers and exploiting them to perform efficient key-recovery attacks with Bayesian optimization, named by BAYESIANKEYSEARCH.

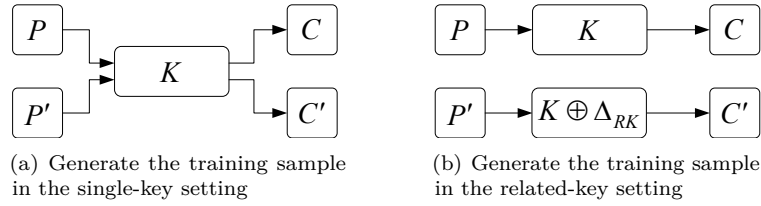


Figure 1: Generating the training sample

2.2.1 Neural-Differential Distinguisher

Training Neural Networks. Given the target cipher $\mathcal{C}_{\mathcal{T}}$ and the fixed input difference Δ_I , the neural network is trained to distinguish between the cipher data and the random data.

As shown in Figure 1(a), the ciphertext pair (C, C') is generated by encrypting (P, P') using the key K . In [Goh19], the training sample only contains the ciphertext pair (C, C') by transforming the ciphertext pair into a quadruple of words. In [BGL⁺22], Bao *et al.* calculate the internal state by partially decrypting the ciphertext pair (C, C') without knowing the last subkey. And the internal state is used as the training sample. The similar cases are also used in [ZWW22, LLS⁺24]. Considering that the derivative of (C, C') – the internal state by partially decrypting the ciphertext pair – is related to the round function of the target cipher and the form of the derivative is not unique, we use (C_{der}, C_{der}') to represent the training sample, even if the new information obtained by the ciphertext pair is added into the training sample.

For a positive sample labeled by 1, the pair of plaintexts (P, P') is encrypted as the training sample using the key K with $P \oplus P' = \Delta_I$. For a negative sample labeled by 0, (P, P') is encrypted using the key K , where the pair of plaintexts is generated randomly.

If the accuracy of the trained network is over 0.5, the trained network is known as the neural distinguisher ($\mathcal{ND}$). Given a new sample (C_{der}, C_{der}') generated by a pair of ciphertexts (C, C') , the $\mathcal{ND}$ will output a score v , where $0 \leq v \leq 1$. If $v > 0.5$, the corresponding ciphertext pair (C, C') is considered to use Δ_I as the input difference;

otherwise, the pair of ciphertexts are considered to use a random difference as the input difference.

In [BLYZ23], Bao *et al.* proposed the neural-differential distinguisher in the related-key setting. Different from the neural-differential distinguisher in the single-key setting shown in [Goh19], as shown in Figure 1(b), the training sample is generated using the ordered quadruple (P, P', K, K') with $K' = K \oplus \Delta_{RK}$. The plaintext P (resp. P') is encrypted to C (resp. C') using the key K (resp. K'). For the positive sample, $P \oplus P' = \Delta_I$. For a negative sample, the difference $P \oplus P'$ is a random value.

Combine-Response Distinguisher and Neutral Bit. As a new technique different from previous distinguishers, there are some limitations in using $\mathcal{ND}$ s. When the accuracy becomes marginally higher than 0.5, it is hard to use in key-recovery attacks.

$$v_{crd} = \sum_{i=1}^t \log_2 \left(\frac{v_i}{1 - v_i} \right) \quad (1)$$

Thus, Gohr uses the combined response (see Equation 1) of the $\mathcal{ND}$ over large number of samples of the same distribution as a distinguisher (named as combined-response-distinguisher, $\mathcal{CRD}$). When using t samples following the same distribution, the response of $\mathcal{CRD}$, v_{crd} , is calculated by Equation 1. And if $v_{crd} > 0$, then these samples are considered to use Δ_{in} as the input difference, else these samples are considered to use a random difference as the input difference. More details of $\mathcal{CRD}$ can be found in the appendix of [BGL⁺21].

It is noticed that it is a necessary condition for $\mathcal{CRD}$ to use the samples following the same distribution. However, it is difficult to obtain a large number of samples following the same distribution due to using probabilistic prepended classical distinguishers in the key-recovery attacks. To this, in [Goh19], Gohr used multiple neutral bits to generate the ciphertext structures, the set of ciphertext pairs following the same distribution. Further, Bao *et al.* [BGL⁺22] focus on finding various types of neutral bits to find enough neutral bits to prepend a long $\mathcal{CD}$ over a weak $\mathcal{ND}$.

Definition 1 (Conforming Pair [BGL⁺22]). Let $\delta := \Delta_{in} \rightarrow \Delta_{out}$ be a differential of an r -round encryption F^r . Let (P, P') be the input pair and (C, C') be the output pair encrypted by F^r , where $P \oplus P' = \Delta_{in}$. If $C \oplus C' = \Delta_{out}$, (P, P') is said to be **Conforming Pair**.

Definition 2 (Neutral bit of a differential, NB[BC04]). Let $\delta := \Delta_{in} \rightarrow \Delta_{out}$ be a differential of an r -round encryption F^r . And e_i is one of the standard basis of $\mathbb{F}_2^n$, where the i -th bit is a non-zero bit. The i -th bit is a neutral bit of $\delta := \Delta_{in} \rightarrow \Delta_{out}$, if for any conforming pair (P, P') , $(P \oplus e_i, P' \oplus e_i)$ is also a conforming pair.

Definition 3 (Simultaneous-Neutral bit-sets, SNBSs [BC04]). Let $\delta := \Delta_{in} \rightarrow \Delta_{out}$ be a differential of an r -round encryption F^r . And e_i is one of the standard basis of $\mathbb{F}_2^n$, where the i -th bit is a non-zero bit. Denote $f_{I_s} = \bigoplus_{i \in I_s} e_i$. The I_s is a simultaneous-neutral bit-sets of $\delta := \Delta_{in} \rightarrow \Delta_{out}$, if for any conforming pair (P, P') , $(P \oplus f_{I_s}, P' \oplus f_{I_s})$ is also a conforming pair, while for any subsets I'_s of I_s , $(P \oplus f_{I'_s}, P' \oplus f_{I'_s})$ is not always a conforming pair.

Further, the NB and the SNBSs in the related-key setting are given in Definition 5 and Definition 6.

Definition 4 (Conforming Pair in the related-key setting). Let $\delta := \Delta_{in} \rightarrow \Delta_{out}$ be a differential of an r -round encryption F^r in the related-key setting. The (P, P') are encrypted r rounds to (C, C') using the key k and $k \oplus \Delta_{rk}$, where $P \oplus P' = \Delta_{in}$. If $C \oplus C' = \Delta_{out}$, (P, P') is said to be **Conforming Pair in the related-key setting**.

Definition 5 (Neutral bit of a differential in the related-key setting). Let $\delta := \Delta_{in} \rightarrow \Delta_{out}$ be a differential of an r -round encryption F^r in the related-key setting. And e_i is one of the standard basis of $\mathbb{F}_2^n$, where the i -th bit is a non-zero bit. The i -th bit is a neutral bit of $\delta := \Delta_{in} \rightarrow \Delta_{out}$, if for any conforming pair (P, P') , $(P \oplus e_i, P' \oplus e_i)$ is also a conforming pair, where P and $P \oplus e_i$ are encrypted using k and P' and $P' \oplus e_i$ are encrypted using $k \oplus \Delta_{rk}$.

Definition 6 (Simultaneous-Neutral bit-sets in the related-key setting). Let $\delta := \Delta_{in} \rightarrow \Delta_{out}$ be a differential of an r -round encryption F^r . And e_i is one of the standard basis of $\mathbb{F}_2^n$, where the i -th bit is a non-zero bit. Denote $f_{I_s} = \bigoplus_{i \in I_s} e_i$. The I_s is a simultaneous-neutral bit-sets of $\delta := \Delta_{in} \rightarrow \Delta_{out}$, if for any conforming pair (P, P') , $(P \oplus f_{I_s}, P' \oplus f_{I_s})$ is also a conforming pair, while for any subsets I'_s of I_s , $(P \oplus f_{I'_s}, P' \oplus f_{I'_s})$ is not always a conforming pair, where P and $P \oplus f_{I_s}$ are encrypted using k and P' and $P' \oplus f_{I_s}$ are encrypted using $k \oplus \Delta_{rk}$.

Algorithm 1 BAYESIANKEYSEARCH Algorithm [Goh19]

Input: Ciphertext Structure: $C_{structure} := \{C_0, C_1, \dots, C_{n_b-1}\}$
 Neural Distinguisher: $\mathcal{ND}$
 Wrong Key Response Profile (WKR $\mathcal{P}$): μ and σ

Output: The list L of tuples of recommended keys and their scores

- 1: $CK := \{k_0, k_1, \dots, k_{Cand-1}\} \leftarrow$ randomly choose $Cand$ values without replacement
- 2: $L \leftarrow \{\}$
- 3: $Loop \leftarrow 0$
- 4: **while** $Loop < Step$ **do**
- 5: $Loop \leftarrow Loop + 1$
- 6: **for** $i \in \{0, 1, \dots, Cand - 1\}$ **do**
- 7: **for** $j \in \{0, 1, \dots, n_b - 1\}$ **do**
- 8: $\bar{C}_{i,j} \leftarrow Decrypt_{1r}(C_j, k_i)$
- 9: $v_{i,j} \leftarrow \mathcal{ND}(\bar{C}_{i,j})$
- 10: $s_{i,j} \leftarrow \log_2(v_{i,j}/(1 - v_{i,j}))$
- 11: **end for**
- 12: $s_{k_i} = \sum_{j=0}^{n_b-1} s_{i,j}$
- 13: $L \leftarrow L \parallel (k_i, s_{k_i})$
- 14: $m_{k_i} = \sum_{j=0}^{n_b-1} v_{i,j} \Big/ n_b$
- 15: **end for**
- 16: **for** $k \in \{0, 1, \dots, 2^{16} - 1\}$ **do**
- 17: $\lambda_k = \sum_{i=0}^{Cand-1} (m_{k_i} - \mu_{k_i \oplus k})^2 \Big/ \sigma_{k_i \oplus k}^2$
- 18: **end for**
- 19: $CK \leftarrow \text{argsort}_k(\lambda)[0 : Cand - 1]$ /* Pick $Cand$ new candidates with the $Cand$ smallest score λ_k . */
- 20: **end while**
- 21: **Return** L

2.2.2 Bayesian Optimization for Key Search

An important detail in the key-recovery attack using neural distinguishers is employing Bayesian optimization to perform a highly selective key search policy, which is named by BAYESIANKEYSEARCH and allows the adversary to use fewer candidates. As shown

in Algorithm 1, the ciphertext structure, the set of ciphertext pairs following the same distribution, consisting of n_b pairs of ciphertexts is decrypted using $Cand \times Step$ candidates. During the initialization phase, $Cand$ subkeys are randomly chosen from the set of all subkey candidates. During the searching phase, these chosen subkeys are used to decrypt the ciphertext structure one round and scored, the s_{k_i} is used as the score of k_i . Considering the relationship between the scores of these chosen subkeys and Wrong Key Response Profile ($WKR\mathcal{P}$, see Appendix B), $Cand$ new candidates are selected to decrypt the ciphertext structure in the next loop until $Cand \times Step$ candidates have been used. With the BAYESIANKEYSEARCH algorithm, an attacker can recover the real subkey using a small set of candidates instead of using all subkey candidates, which improves the time complexity by reducing the number of used candidates. However, this calls into the next question. How to achieve the balance between the number of candidates $Cand \times Step$ and the success rates of key-recovery attacks? And how to assign the values of $Cand$ and $Step$ to reach the full potential of $\mathcal{ND}$ and the Bayesian optimization?

2.2.3 Framework of the Key-Recovery Attack

Using the Bayesian-optimized key-guessing strategy, the cornerstone of the key-recovery attack, Gohr applied the strategy two times to two different neural distinguishers to recover the last two subkeys. As illustrated in Figure 2, given a ciphertext structure $C_{structure}$, one decrypts it using $Cand_1 \times Step_1$ candidates for the last subkey, denoted as k_{-1} . One performs the first BAYESIANKEYSEARCH (denoted as BKS_1) to recover k_{-1} . Candidates of k_{-1} with scores exceeding the cutoff c_1 are forwarded to the next stage, where one attempts to recover the second-to-last subkey k_{-2} using the second BAYESIANKEYSEARCH. If no candidate achieves a score greater than c_1 , one proceeds to the next ciphertext structure.

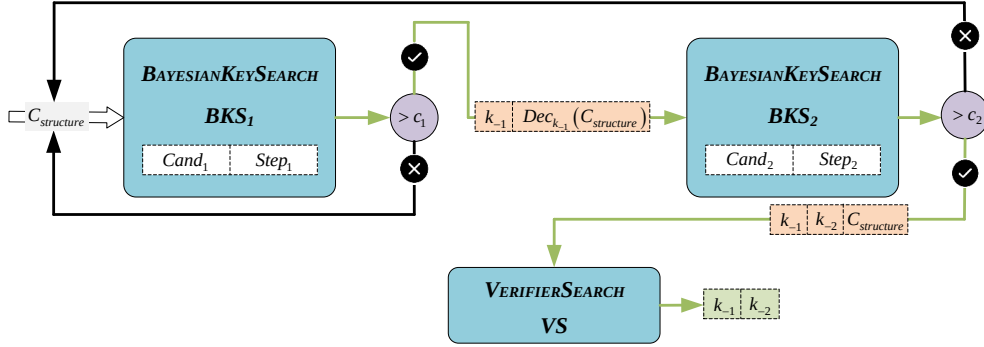


Figure 2: Components of employing BAYESIANKEYSEARCH two times

Using the selected candidates for k_{-1} and the ciphertext structure decrypted by one round using these candidates, one performs the second BAYESIANKEYSEARCH to recover k_{-2} . If any candidate for k_{-2} has a score exceeding the cutoff c_2 , one applies a final improvement using VERIFIERSEARCH (denoted as VS) to correct any incorrect bits in k_{-1} and k_{-2} . Otherwise, one selects the next ciphertext structure and repeats BKS_1 . In [Goh19], Gohr designed an effective structure selection policy to select the next ciphertext structure with high potential, based on a standard exploration-exploitation technique, namely Upper Confidence Bounds (UCB).

The key-recovery attack continues until either a ciphertext structure reaches the VS stage or the number of tested ciphertext structures reaches a predefined limit.

The use of BAYESIANKEYSEARCH reduces the subkey candidates space, improving time complexity. However, due to the prepended classical distinguisher $\mathcal{CD}$ having a

probability less than 1, not all ciphertext structures are effective for key-recovery using BAYESIANKEYSEARCH. Therefore, it is indispensable to investigate how to tune the six parameters of BKS_1 and BKS_2 : $(Cand_1, Step_1, c_1, Cand_2, Step_2, c_2)$, to select appropriate structures and enhance the key-recovery attack.

2.3 Related Works

In ISC 2022, Lyu *et al.* [LTZ22] listed some of their experience in parameter tuning. Based on empirical observations, they noted that the choice of cutoffs significantly influences both the success rate and the time complexity of the attack. However, only qualitative analysis was performed on the relationship between cutoff values, success rate, and time complexity. Additionally, the authors provided no clear guidance for adjusting the two cutoffs, proposing instead that adjustments be made based on specific experimental results. Notably, they failed to illustrate the adjustment of cutoffs based on experimental results with a concrete example, even if they performed the key-recovery attacks on 13/14/15-round SIMECK32/64 [YZS⁺15]. And a uniform cutoff value of 10.0 was applied across all key-recovery attacks on SIMECK32/64, covering 13, 14, and 15 rounds.

Different from [LTZ22], Bao *et al.* [BGL⁺22] provided detailed experimental data and derived observations to bring some light on tuning important parameters and making better trade-offs. Bao *et al.* [BGL⁺22] proposed that the value of c_1 was determined by observing the distribution of the output of the BAYESIANKEYSEARCH algorithm. And a formula was presented to calculate the upper bound on the value of the cutoff c_1 . Unlike the detailed description for the selection of c_1 , the authors concluded that a value for c_2 merely needed to be sufficiently large. Notably, the two cutoffs are determined independently.

In addition to the work on tuning important parameters of neural-differential attack, the researcher has explored to optimize the performance of BAYESIANKEYSEARCH. Zheng *et al.* [ZZW24] proposed to generate new key candidates using the retaining key candidates. Given a ciphertext structure $C_{structure}$, if it is processed as input to the operation illustrated in Figure 2 for the first time, the initial *Cand* candidates in Step 1 of Algorithm 1 are generated randomly. Upon the completion of the Algorithm 1 execution, the top-*Cand* highest-scoring candidates are preserved. When the same ciphertext structure is again supplied as input to the operation depicted in Figure 2, the saved candidates are selected in the first step of Algorithm 1 instead of generating new candidates randomly. However, the improvements in computational time and success rate were ultimately found to be insignificant. Along with this, Zheng *et al.* introduced the improved selection strategy of ciphertext structure to select promising structures as the input of Figure 2, which improved time complexity and success rate.

As illustrated in Figure 2, the cutoffs are applied to filter the candidates, advancing those exceeding the score requirement to the subsequent stage. A lower value of c_1 increases candidate volume (at most $Cand_1 \times Step_1$), which serves as input to BKS_2 . Furthermore, the total running time of executing BKS_2 is increased. However, a higher value of c_1 reduces the probability that the correct guesses are forwarded to the next stage, which further reduces the success rate. For c_2 selection, higher values reduce the probability of correct guesses being forwarded to *VS* while increasing the likelihood of progressing to subsequent ciphertext structures. And lower c_2 values increases the probability that the wrong guesses are used as the output of the key recovery. Thus, the lower bound for c_2 must be established. Considering that the lower bound for c_2 is influenced by the quantity of wrong candidates, the second cutoff should be decided alongside the first.

3 Improved Related-Key Differential-Neural Distinguisher

Previous works have shown that using more ciphertext data in the single training sample helps the neural networks reach the higher accuracy of distinguishing cipher data from random data. But the method injecting more ciphertexts into the training samples cannot achieve the better distinguishing effect when the number of plaintexts is limited. Using the same size of plaintexts, most of them can't reach the baseline of the combined-response distinguishers (CRD). Thus, it is significant to offer more or more obvious features for the neural networks using the same size of plaintexts. In [BGL⁺22], Bao *et al.* added the differences of the penultimate round into the training samples, which achieved slightly better accuracy than only using two 32-bit ciphertexts as the training sample. Bao *et al.*'s work revealed that adding some explicit features without using more plaintexts (the differences of the penultimate and antepenultimate rounds, for instance) is effective for improving the $\mathcal{ND}$.

In this section, we introduce a new format of the training sample for related-key differential-neural distinguishers, aiming to help the neural networks obtain more and more obvious features without utilizing more plaintexts, which enhances the effect of distinguishing cipher data from random data.

3.1 New Format for Related-Key Differential-Neural Distinguisher

In the following, we start with two symbols to represent the features contained by the single-key training sample (see Figure 1(a)) and the related-key training sample (see Figure 1(b)).

Features contained by the single-key training sample. In the single-key setting, a pair of plaintexts (P, P') is encrypted by a master key K to the ciphertext pair $(E_K(P), E_K(P'))$, where the $E_K(P)$ (resp. $E_K(P')$) denotes encrypting P (resp. P') with the master key K . The adversary trains the neural networks to capture the feature used to distinguish cipher data from random data. Denote the set of the features captured by the neural networks by F^{single} .³ It is noticed that a training sample built from a pair of ciphertexts in the single-key setting only has one F^{single} .

Features contained by the related-key training sample. In the related-key setting, a pair of plaintexts (P, P') is encrypted by a pair of master keys (K, K') to the ciphertext pair $(E_K(P), E_{K'}(P'))$, where $K \oplus K' = \Delta_{RK}$. Denote the set of all features used in the neural networks in the related-key setting by $F^{related}$. And a training sample using a pair of ciphertexts shown in Figure 1(b) only has one $F^{related}$.

Not limited to only obtaining a pair of ciphertext in the related-key setting, as shown in Figure 3, each of plaintexts can be encrypted two times using the different master keys with reusing the plaintext pair, where $P \neq P'$, $C_0 = E_K(P)$, $C_1 = E_K(P')$, $C_2 = E_{K \oplus \Delta_{RK}}(P)$, $C_3 = E_{K \oplus \Delta_{RK}}(P')$.

It is obvious that there is one set of features in the single-key setting or the related-key setting. In other words, each training sample, using the data format shown in Figure 1, is allowed to have one set of features. Compared with previous works, with four ciphertexts shown in Figure 3, there are six sets of features in total, two F^{single} s from (C_0, C_1) and (C_2, C_3) , and four $F^{related}$ s from (C_0, C_2) , (C_1, C_3) , (C_0, C_3) and (C_1, C_2) . Utilizing this method, a training sample can contain more features without using more plaintexts.

³In [BGPT21], Benamira *et al.* show with experiments that the feature captured neural networks is related to the truncated differential distribution. And in [BLYZ23], Bao *et al.* further show that the differential-neural distinguisher may use additional information besides pure ciphertext differences. Thus, the F^{single} is used to refer to the all features of one pair of ciphertexts in the single-key setting, where there are the features from the differential distribution and potential unknown features in F^{single} .

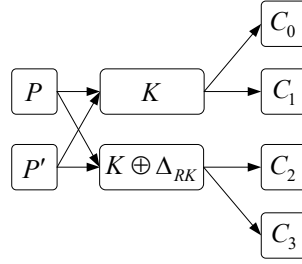


Figure 3: Reusing the plaintexts in the related-key setting

3.2 Application to SIMON32/64

Here, we present the effectiveness of the new training data format on the related-key differential-neural distinguishers of r -round SIMON32/64, where the r -round distinguisher is denoted by $\mathcal{RK}\text{-}\mathcal{ND}^{\text{SIMON}_{rR}}$.

The Choice of the Input Difference and the Master Key Difference. Considering that there are two sets of features from the ciphertexts pair in the single-key setting, we choose $(0x0, 0x40)$ as the input difference, which is verified to be effective in training neural networks by experiments in [BGL⁺22, GLN22, LLS⁺24]. With the input difference $(0x0, 0x40)$, $(0x0, 0x0, 0x0, 0x40)$ is employed as the key difference. For the ciphertext pairs (C_0, C_2) and (C_1, C_3) obtained by encrypting the same plaintext two times using the different master keys, the state difference is influenced by the first subkey and the state differences in subsequent rounds are independent of the subkeys until the subkey difference becomes nonzero again. For the ciphertext pairs (C_0, C_3) and (C_1, C_2) obtained by the plaintext pairs with $\Delta_I = (0x0, 0x40)$, the state differences in the first four rounds hold zero with the master key difference $(0x0, 0x0, 0x0, 0x40)$, which helps to maximize the number of rounds for neural distinguishers.

Training Using the New Format. For the positive samples, the plaintext difference $P \oplus P'$ is given $(0x0, 0x40)$. For the negative samples, the differences are generated randomly. Given the 6 pairs of ciphertexts (C_0, C_1) , (C_2, C_3) , (C_0, C_2) , (C_1, C_3) , (C_0, C_3) and (C_1, C_2) , we consider of the extensions for each of ciphertext pairs. The 384-bit vector $f(C_0, C_1) || f(C_2, C_3) || f(C_0, C_2) || f(C_1, C_3) || f(C_0, C_3) || f(C_1, C_2)$ is used the training samples, where $f(X, X') = X \oplus X' || A \oplus A' || f_{\text{round}}(A) \oplus f_{\text{round}}(A') \oplus X_R \oplus X'_R$, $A = f_{\text{round}}(X_R) \oplus X_L$, $A' = f_{\text{round}}(X'_R) \oplus X'_L$, $f_{\text{round}}(x) = (x \lll 1 \& x \lll 8) \oplus x \lll 2$, $X = X_L || X_R$ and $X' = X'_L || X'_R$. In [BGL⁺22], Bao *et al.* added the difference of the penultimate round of form $f_{\text{round}}(X_R) \oplus X_L \oplus f_{\text{round}}(X'_R) \oplus X'_L$ in the training sample to improve the neural networks. And in [LLS⁺24], Lu *et al.* further offered more information of form $f_{\text{round}}(f_{\text{round}}(X_R) \oplus X_L) \oplus f_{\text{round}}(f_{\text{round}}(X'_R) \oplus X'_L) \oplus X_R \oplus X'_R$ to further enhance the capacity of distinguishing cipher data from random data. In the new format for the related-key neural distinguishers of round-reduced SIMON32/64, we employ these two methods to help the neural networks to capture these features from multiple ciphertext pairs.

Training Using Multiple Network Architectures. Given the compromise between the computing overhead and the accuracy of $\mathcal{RK}\text{-}\mathcal{ND}$, we first train the short-round $\mathcal{RK}\text{-}\mathcal{ND}$ s using the residual neural network used in [Goh19]. When the residual neural network fails, a more complex network consisting of the freezing layer [YLL⁺22] and the Inception block [SLJ⁺15] is employed. The best 11-round and 12-round distinguishers are trained using the residual neural network. The 13-round distinguisher is trained using the complex network.

Training Using the Staged Training Method. In [Goh19], there are three stages to train an 8-round distinguisher of SPECK32/64, and the size of training samples and the learning rate are tuned in each stage. In this section, there are only two stages to train

$\mathcal{RK}\text{-}\mathcal{ND}^{\text{SIMON}_{13R}}$. In the first stage, the $\mathcal{RK}\text{-}\mathcal{ND}^{\text{SIMON}_{12R}}$ is retained using the complex network consisting of the freezing layer and the Inception block. In the second stage, the $\mathcal{RK}\text{-}\mathcal{ND}^{\text{SIMON}_{13R}}$ is trained using the parameters of $\mathcal{RK}\text{-}\mathcal{ND}^{\text{SIMON}_{12R}}$ trained in the first stage.

Training Setup. In the basic training scheme, used to train 11- and 12-round neural distinguishers, we run the training for 50 epochs on the training set and the validation set, which includes 2×10^7 samples and 2×10^6 samples, respectively. We set the batch size to 10000. A linear learning rate schedule was applied, setting the learning rate l_i for epoch i to $l_i = 2 \times 10^{-3} - \frac{2 \times 10^{-3} - 2 \times 10^{-4}}{50} \times i$. In the staged training scheme, we retrain the 12-round neural distinguisher using the same hyper-parameters as the basic scheme. The 13-round distinguisher is trained by using the network weight of the 12-round distinguisher. We run the training for 150 epochs on the training set and the validation set, which includes 2×10^7 samples and 2×10^6 samples, respectively. The training samples are re-generated during each epoch. A linear learning rate schedule was applied five times, setting the learning rate l_i for epoch i to $l_i = 3.5 \times 10^{-3} - \frac{3.5 \times 10^{-3} - 2 \times 10^{-4}}{30} \times (i \bmod 30)$. Throughout the training phase, both training and validation set accuracy metrics are systematically recorded at each epoch's conclusion to comparatively assess model performance and detect potential overfitting tendencies.

We summarize the accuracy of the related-key differential-neural distinguishers in Table 2. To the best of our knowledge, there are state-of-the-art related-key neural distinguishers of SIMON32/64.

Table 2: The summary of the related-key differential-neural distinguishers on SIMON32/64.

#R	Type	Accuracy	True Positive Rate	True Negative Rate	Reference
11	$\mathcal{RK}\text{-}\mathcal{ND}$	0.6936 ^{*⊗}	0.7398	0.6472	[LLS ⁺ 24]
11	$\mathcal{RK}\text{-}\mathcal{ND}$	0.7829 [*]	0.8029	0.7628	Section 3
11	$\mathcal{RK}\text{-}\mathcal{ND}$	0.9381 ^{†⊗}	0.9441	0.9321	[LLS ⁺ 24]
11	$\mathcal{RK}\text{-}\mathcal{ND}$	0.9885 [†]	0.9895	0.9874	Section 3
11	$\mathcal{RK}\text{-}\mathcal{ND}$	0.9604 [‡]	0.9639	0.9569	[LLS ⁺ 24]
11	$\mathcal{RK}\text{-}\mathcal{ND}$	0.9753 [‡]	0.9779	0.9727	Section 3
12	$\mathcal{RK}\text{-}\mathcal{ND}$	0.5424 ^{*⊗}	0.5870	0.4977	[LLS ⁺ 24]
12	$\mathcal{RK}\text{-}\mathcal{ND}$	0.5728 [*]	0.5819	0.5637	Section 3
12	$\mathcal{RK}\text{-}\mathcal{ND}$	0.6332 ^{†⊗}	0.6490	0.6174	[LLS ⁺ 24]
12	$\mathcal{RK}\text{-}\mathcal{ND}$	0.7032 [†]	0.7013	0.7050	Section 3
12	$\mathcal{RK}\text{-}\mathcal{ND}$	0.6477 [‡]	0.6518	0.6435	[LLS ⁺ 24]
12	$\mathcal{RK}\text{-}\mathcal{ND}$	0.6717 [‡]	0.6804	0.6631	Section 3
13	$\mathcal{RK}\text{-}\mathcal{ND}$	0.5011 ^{*⊗}	0.1378	0.8645	[LLS ⁺ 24]
13	$\mathcal{RK}\text{-}\mathcal{ND}$	0.5120 [*]	0.5090	0.5149	Section 3
13	$\mathcal{RK}\text{-}\mathcal{ND}$	0.5074 ^{†⊗}	0.3879	0.6269	[LLS ⁺ 24]
13	$\mathcal{RK}\text{-}\mathcal{ND}$	0.5307 [†]	0.5448	0.5167	Section 3
13	$\mathcal{RK}\text{-}\mathcal{ND}$	0.5262 [‡]	0.5437	0.5081	[LLS ⁺ 24]
13	$\mathcal{RK}\text{-}\mathcal{ND}$	0.5286 [‡]	0.5427	0.5145	Section 3

* The accuracy is calculated using the single samples consisting of two plaintexts.

⊗ The accuracy is calculated using the source code of [LLS⁺24].

† The accuracy is calculated using the combine-response distinguisher and 16 plaintexts.

‡ The accuracy is calculated using the single samples consisting of 16 plaintexts.

- When generating one sample using the same number of plaintexts, the number of the encryption queries using the new format is there are twice as many in using the new format as in [LLS⁺24].

4 Prepended Classical Distinguishers

In this paper, we show an 18-round related-key practical attack against SIMON32/64 by employing the 12-round and the 13-round related-key neural distinguishers in Section 3, shown in Figure 5. The 12-round and 13-round distinguishers are employed to recover the last subkey and the second to last subkey. A 3-round classical distinguisher ($\mathcal{CD}$) is used to improve the number of rounds of the key-recovery attacks. In this section, we mainly analyze the prependded 3-round differential distinguishers in the single-key and related-key settings.

Different from using one classical distinguisher in the single-key setting, there needs four prependded differential characteristics to meet the requirements of the $\mathcal{RK}\text{-}\mathcal{ND}$, which increases the data complexity. Fortunately, there is a relation in these differential characteristics between the single-key setting and the related-key setting. Based on the relation, we precisely describe the differential propagations in the prependded classical distinguishers. Further cryptanalysis is carried out to build the constraints for plaintexts.

4.1 Basic Description

Considering only the linear function in the key schedule, the subkey differences in the first four rounds are $(0x4, 0x74, 0x50, 0x60)$. The subkey difference in the first round (the free round) is $\Delta_{k_0} = 0x4$. And the subkey differences in the rounds using the classical distinguishers are $(\Delta_{k_1}, \Delta_{k_2}, \Delta_{k_3}) = (0x74, 0x50, 0x60)$.

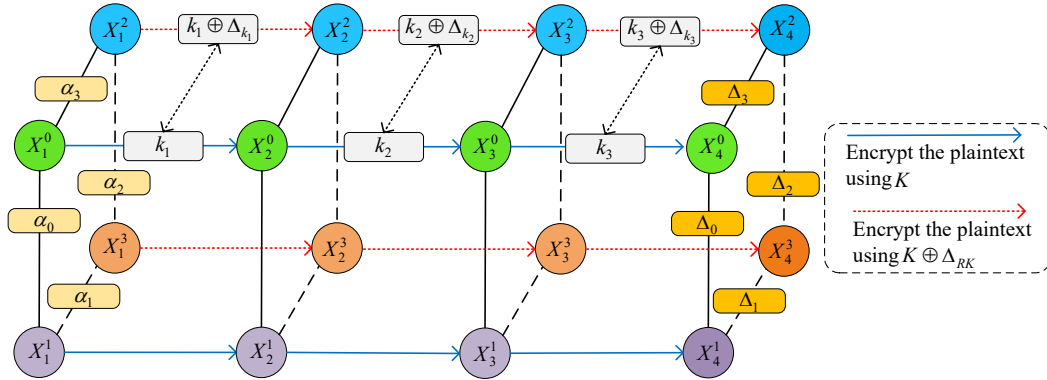


Figure 4: 3-round classical distinguishers in the single-key and related-key setting

As shown in Figure 4, with the input difference of $\mathcal{RK}\text{-}\mathcal{ND}$, $\Delta_0 = \Delta_2 = (0x0, 0x40)$, $\Delta_1 = \Delta_3 = (0x0, 0x0)$. The state X_1^0 and X_1^1 are encrypted to X_4^0 and X_4^1 using the subkey k_1 , k_2 and k_3 . And the state X_1^2 and X_1^3 are encrypted to X_4^2 and X_4^3 using the subkey $k_1 \oplus 0x74$, $k_2 \oplus 0x50$ and $k_3 \oplus 0x60$.

4.2 Searching for the Differential Characteristics

For the differential characteristic in the single-key setting, with the help of an automatic tool, the optimal 3-round differential characteristic using $(0x0, 0x40)$ as the output difference is

$$(0x440, 0x1000) \xrightarrow{2^{-4}} (0x100, 0x440) \xrightarrow{2^{-2}} (0x40, 0x100) \xrightarrow{2^{-2}} (0x0, 0x40).$$

Besides, there are three other differential characteristics using $(0x440, 0x1000)$ and $(0x0, 0x40)$ as the input and the output differences with the differential probability 2^{-10} , 2^{-10} and 2^{-11} , respectively. With the four differential characteristics, the difference $(0x440, 0x1000) \rightarrow$

$(0x0, 0x40)$ holds $\Pr((0x440, 0x1000) \rightarrow (0x0, 0x40)) \geq 2^{-7.23}$. Given the input and output differences $(0x440, 0x1000)$ and $(0x0, 0x40)$, an exhaustive search is performed to search for simultaneous-neutral bit-sets (SNBSs) by considering 5488 sets (*i.e.*, $32 + 496 + 4960$ sets) of at most three bits. There are 9 single-bit NBs $\{[2], [3], [4], [6], [8], [9], [10], [18], [22]\}$, 2 SNBSs $\{[0, 24], [12, 26]\}$.

For the differential characteristic in the related-key setting, an optimal characteristic is obtained with the differential probability 2^{-10} . With the optimal characteristic

$$(0x1d0, 0x754) \xrightarrow[\Delta_{k_1}=0x74]{2^{-4}} (0x60, 0x1d0) \xrightarrow[\Delta_{k_2}=0x50]{2^{-4}} (0x0, 0x60) \xrightarrow[\Delta_{k_3}=0x60]{2^0} (0x0, 0x0),$$

an exhaustive search is performed to search for SNBSs by considering of 5488 sets (*i.e.*, $32 + 496 + 4960$ sets) of at most three bits. There are 9 single-bit NBs $\{[2], [3], [4], [6], [7], [8], [9], [20], [22]\}$, 3 SNBS $\{[10, 24], [12, 26], [11, 18, 25]\}$.

Given the differentials in the single-key setting and the related-key setting, the ordered quadruple $(X_1^0, X_1^1, X_1^2, X_1^3)$ is generated with $\alpha_0 = \alpha_2 = (0x440, 0x1000)$, $\alpha_1 = \alpha_3 = (0x1d0, 0x754)$. And with $\Delta_2 = \Delta_0 \oplus \Delta_1 \oplus \Delta_3$, we only take into account 3 differentials $\alpha_0 \rightarrow \Delta_0$, $\alpha_1 \rightarrow \Delta_1$ and $\alpha_3 \rightarrow \Delta_3$. In the following, we will demonstrate that it is feasible by only considering of two differentials $\alpha_0 \rightarrow \Delta_0$ and $\alpha_3 \rightarrow \Delta_3$.

4.3 More Details of the 3-round Classical Distinguisher

Given the three differentials $\alpha_0 \rightarrow \Delta_0$, $\alpha_1 \rightarrow \Delta_1$ and $\alpha_3 \rightarrow \Delta_3$ and the ordered quadruple $(X_1^0, X_1^1, X_1^2, X_1^3)$, if these differentials are separate, the probability holding $X_4^0 \oplus X_4^1 = (0x0, 0x40)$, $X_4^0 \oplus X_4^2 = (0x0, 0x0)$, $X_4^1 \oplus X_4^3 = (0x0, 0x0)$ is $2^{-10} \times 2^{-10} \times 2^{-7.3} = 2^{-27.3}$, which makes the key-recovery attacks impossible with the data complexity.

In the following, we give the specific value of $\Pr((X_1^0, X_1^1, X_1^2, X_1^3) \xrightarrow{3r} (X_4^0, X_4^1, X_4^2, X_4^3))$. $\Pr((X_1^0, X_1^1, X_1^2, X_1^3) \xrightarrow{3r} (X_4^0, X_4^1, X_4^2, X_4^3))$ denotes the probability of that the ordered quadruple $(X_4^0, X_4^1, X_4^2, X_4^3)$ satisfies $X_4^0 \oplus X_4^1 = (0x0, 0x40)$, $X_4^0 \oplus X_4^2 = (0x0, 0x0)$, $X_4^1 \oplus X_4^3 = (0x0, 0x0)$, given $X_1^0 \oplus X_1^1 = (0x440, 0x1000)$, $X_1^0 \oplus X_1^2 = (0x1d0, 0x754)$, $X_1^1 \oplus X_1^3 = (0x1d0, 0x754)$.

Property 1. Using the ordered quadruple $(X_1^0, X_1^1, X_1^2, X_1^3)$ as the plaintexts, the quadruple $(X_1^0, X_1^1, X_1^2, X_1^3)$ is encrypted three rounds to $(X_4^0, X_4^1, X_4^2, X_4^3)$ using K and $K \oplus \Delta_{RK}$ as the master keys, where $X_1^0 \oplus X_1^1 = (0x440, 0x1000)$, $X_1^0 \oplus X_1^2 = (0x1d0, 0x754)$, $X_1^1 \oplus X_1^3 = (0x1d0, 0x754)$. The first two elements in $(X_1^0, X_1^1, X_1^2, X_1^3)$ are encrypted using K as the master key. And the last two elements are encrypted using $K \oplus \Delta_{RK}$ as the master key. And the subkey differences are $(\Delta_{k_1}, \Delta_{k_2}, \Delta_{k_3}) = (0x74, 0x50, 0x60)$. The pair of ciphertexts (X_4^0, X_4^2) holds $X_4^0 \oplus X_4^2 = 0x0$, if and only if (X_4^1, X_4^3) holds $X_4^1 \oplus X_4^3 = 0x0$.

Proof. It is obvious that $(X_1^1, X_1^3) = (X_1^0 \oplus (0x440, 0x1000), X_1^2 \oplus (0x440, 0x1000))$. And for the difference $(0x440, 0x1000)$, the locations of non-zero bits are $[12, 22, 26]$. Considering of the differential in the related-key setting,

$$(0x1d0, 0x754) \xrightarrow[\Delta_{k_1}=0x74, \Delta_{k_2}=0x50, \Delta_{k_3}=0x60]{3rounds} (0x0, 0x0),$$

it can be found that $[22]$ and $[12, 26]$ are the neutral bits of the differential. Thus, the property is proved. $\square$

Based on **Property 1**, we only consider of two differentials $\alpha_0 \rightarrow \Delta_0$ and $\alpha_3 \rightarrow \Delta_3$. In the following, we further research how to build the state X_1^0 to meet $X_4^0 \oplus X_4^2 = (0x0, 0x0)$ and $X_4^0 \oplus X_4^1 = (0x0, 0x40)$.

Given the input and output differences in the single-key setting $(0x440, 0x1000)$ and $(0x0, 0x40)$, one can build the non-linear equations to describe the relationship between

each of the plaintext bits and each of the subkey bits. Similarly, with the input and output differences in the related-key setting $(0x1d0, 0x754)$ and $(0x0, 0x0)$, another non-linear equation can be built to describe the relationship between each plaintext bit and each of the subkey bits. A union of non-linear equations is built by merging the non-linear equations in the single-key setting and the related-key setting. Because the degrees of the two 3-round differentials are low, the union can be solved by computing the reduced Gröbner basis, which is shown in Equation 2.

There are 25 variables in total in the Equation 2, where 17 variables are related to the state bits $X_1^0[1, 11, 12, 13, 14, 15, 16, 17, 19, 21, 25, 26, 27, 28, 29, 30, 31]$ and 8 variables are related to the subkey bits $k_1[1, 11, 12, 13, 14, 15]$, $k_2[13, 15]$. Further, it is verified by experiments that the equations shown in Equation 2 have 2^4 and only 2^4 solutions for the values of $X_1^0[1, 11, 12, 13, 14, 15, 16, 17, 19, 21, 25, 26, 27, 28, 29, 30, 31]$ when $k_2[15] = 0$, given each group of specific values of $k_1[1, 11, 12, 13, 14, 15]$ and $k_2[13]$. And when $k_2[15] = 1$, there is no solution for Equation 2.

$$\left\{ \begin{array}{l} X_1^0[16] = 0 \\ X_1^0[17] = 0 \\ X_1^0[19] = 0 \\ X_1^0[27] = 0 \\ X_1^0[29] = 0 \\ X_1^0[30] = 0 \\ X_1^0[31] = 0 \\ X_1^0[25] \oplus X_1^0[11] \oplus k_1[11] \oplus k_2[13] = 0 \\ X_1^0[26] \oplus X_1^0[12] \oplus k_1[12] = 0 \\ X_1^0[28] \oplus X_1^0[14] \oplus k_1[14] = 0 \\ X_1^0[1] \oplus k_1[1] = 0 \\ (X_1^0[21] \odot X_1^0[13]) \oplus (X_1^0[21] \odot k_1[13]) \oplus X_1^0[13] \oplus k_1[13] = 0 \\ (X_1^0[21] \odot X_1^0[14]) \oplus (X_1^0[21] \odot k_1[14]) \oplus X_1^0[13] \oplus k_1[13] = 0 \\ (X_1^0[13] \odot X_1^0[14]) \oplus (X_1^0[13] \odot k_1[14]) \oplus X_1^0[13] \oplus (X_1^0[14] \odot k_1[13]) \oplus (k_1[13] \odot k_1[14]) \oplus k_1[13] = 0 \\ X_1^0[15] \oplus k_1[15] = 0 \\ k_2[15] = 0 \end{array} \right. \quad (2)$$

For 15 bits of X_1^0 unrelated to Equation 2, $X_1^0[0, 2, 3, 4, 5, 6, 7, 8, 9, 10, 18, 20, 22, 23, 24]$, each group of specific values has no influence on making a series of equations sense. Thus, there are $2^{15} \times 2^4 = 2^{19}$ feasible values of X_1^0 , when the master key holds $k_2[15] = 0$. And there are no plaintexts pairs to conform to the two 3-round differentials $(0x440, 0x1000) \rightarrow (0x0, 0x40)$ and $(0x1d0, 0x754) \rightarrow (0x0, 0x0)$, when the master key holds $k_2[15] = 1$.

In other words, $\Pr((X_1^0, X_1^1, X_1^2, X_1^3) \xrightarrow{3r} (X_4^0, X_4^1, X_4^2, X_4^3)) = 2^{-13}$ when the master key holds $k_2[15] = 0$. And $\Pr((X_1^0, X_1^1, X_1^2, X_1^3) \xrightarrow{3r} (X_4^0, X_4^1, X_4^2, X_4^3)) = 0$, when the master key holds $k_2[15] = 1$. Based on this, randomly generating n X_1^0 , with $X_1^0 \oplus X_1^1 = (0x440, 0x1000)$, $X_1^0 \oplus X_1^2 = (0x1d0, 0x754)$, there are $2^{-13} \times n$ values in average to satisfy $X_4^0 \oplus X_4^1 = (0x0, 0x40)$ and $X_4^0 \oplus X_4^2 = (0x0, 0x0)$, when the master key holds $k_2[15] = 0^4$.

Considering the case that not changing the reduced Gröbner basis is a necessary and sufficient condition for being an NB, there are 15 single-bit NBs $\{[0], [2], [3], [4], [5], [6], [7], [8], [9], [10], [18], [20], [22], [23], [24]\}$ to satisfy both of two 3-round differentials $(0x440, 0x1000) \rightarrow (0x0, 0x40)$ and $(0x1d0, 0x754) \rightarrow (0x0, 0x0)$, which are the bit locations of X_1^0 missing from Equation 2.

Due to that the 15 single-bit NBs are not occurred in Equation 2, it is found that the 15 single-bit neutral bits are deterministic. In other words, given two quadruples $(X_1^0, X_1^1, X_1^2, X_1^3)$ and $(X_1^0 \oplus e_i, X_1^1 \oplus e_i, X_1^2 \oplus e_i, X_1^3 \oplus e_i)$, encrypting the two quadruples 3 rounds to generate the quadruple $(X_4^0, X_4^1, X_4^2, X_4^3)$ and $(\overline{X_4^0}, \overline{X_4^1}, \overline{X_4^2}, \overline{X_4^3})$, there are $X_4^0 \oplus \overline{X_4^0} = (0x0, 0x40)$, $X_4^1 \oplus \overline{X_4^1} = (0x0, 0x0)$, $X_4^2 \oplus \overline{X_4^2} = (0x0, 0x0)$ if and only if there are $\overline{X_4^0} \oplus \overline{X_4^1} = (0x0, 0x40)$, $\overline{X_4^0} \oplus \overline{X_4^2} = (0x0, 0x0)$, $\overline{X_4^1} \oplus \overline{X_4^3} = (0x0, 0x0)$, where only $e_i[i]$ is the only non-zero bit of e_i and $i \in \{0, 2, 3, 4, 5, 6, 7, 8, 9, 10, 18, 20, 22, 23, 24\}$.

⁴In the following, we only consider the master key holding $k_2[15] = 0$.

5 Tuning Parameters for BAYESIANKEYSEARCH

Testing multiple parameter configurations of BAYESIANKEYSEARCH to find the optimal choice through extensive attack trials is impractical for key-recovery attacks that employ complex neural networks and large volumes of ciphertexts. To address this challenge, we propose a more practical and efficient approach that combines theoretical analysis with data obtained from experimental evaluations. This method allows for effective parameter optimization without the need for exhaustive trials, thereby shifting the previous paradigm.

As a case study, we use the 18-round attack on SIMON32/64 (illustrated in Figure 5), which employs the enhanced 12- and 13-round related-key neural distinguishers detailed in Section 3. We demonstrate how to select appropriate parameters for BAYESIANKEYSEARCH and determine optimal cutoffs for filtering ciphertext structures. Furthermore, our approach enables us to evaluate a lower bound on the success rate by integrating theoretical analysis with experimental data, reducing reliance on impractical or time-consuming attack trials.

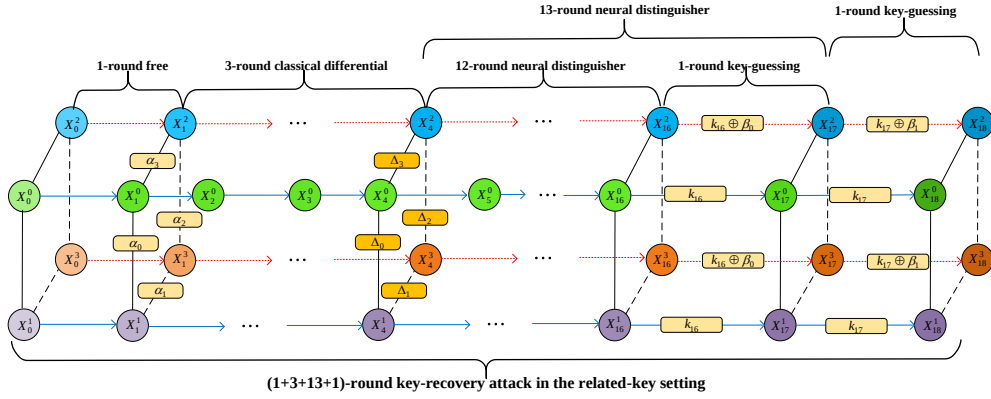


Figure 5: Components of the key-recovery attacks in the related-key setting

5.1 Basic Explanation

Before analyzing the influence of these six parameters ($Cand_1, Step_1, c_1, Cand_2, Step_2, c_2$) on the key-recovery attacks, we provide some basic explanations of the ciphertext structure. There are two types of ciphertext structures: one is the right structure, and another is the wrong structure.

Right Ciphertext Structure (RC_s). The plaintext structure containing 2^{13} plaintext pairs is encrypted 4 rounds to obtain the middle state using K and $K \oplus \Delta_{RK}$. If each quadruple $(X_4^0, X_4^1, X_4^2, X_4^3)$ in the middle state satisfies the conditions $X_4^0 \oplus X_4^1 = (0x0, 0x40)$, $X_4^0 \oplus X_4^2 = (0x0, 0x0)$, and $X_4^1 \oplus X_4^3 = (0x0, 0x40)$, then the plaintext structure is referred to as the right plaintext structure and the corresponding ciphertext structure is called the *Right Ciphertext Structure*.

Wrong Ciphertext Structure (WC_s). The plaintext structure is encrypted for 4 rounds to obtain the middle state structure using subkeys corresponding to K and $K \oplus \Delta_{RK}$. If the three conditions $X_4^0 \oplus X_4^1 = (0x0, 0x40)$, $X_4^0 \oplus X_4^2 = (0x0, 0x0)$ and $X_4^1 \oplus X_4^3 = (0x0, 0x40)$ cannot be simultaneously satisfied by a quadruple $(X_4^0, X_4^1, X_4^2, X_4^3)$ in the middle state structure, then the corresponding plaintext structure is termed a wrong plaintext structure and the corresponding ciphertext structure is called the *Wrong Ciphertext Structure*.

Remark 1. Using the deterministic neutral bits to generate the ciphertext structures, the 2^{13} quadruples $(X_4^0, X_4^1, X_4^2, X_4^3)$ in the middle state structure either all satisfy or all fail to satisfy the three conditions simultaneously: $X_4^0 \oplus X_4^1 = (0x0, 0x40)$, $X_4^0 \oplus X_4^2 = (0x0, 0x0)$, and $X_4^1 \oplus X_4^3 = (0x0, 0x40)$.

With that, the probability of satisfying multiple 3-round classical differentials simultaneously is 2^{-13} . When randomly generating n_{cts} ciphertext structures, the probability that at least one RC_s exists among n_{cts} ciphertext structures is given by $P_{RC_s} = 1 - (1 - 2^{-13})^{n_{cts}}$. In the following analysis, we consider $n_{cts} = 2^{14}$ and assume that there is at least one RC_s in the set.

Given the definitions of the right ciphertext structure and the wrong ciphertext structure, we make the following observation about the relationship between the ciphertext structures and the candidate key guesses with corresponding scores in BAYESIANKEYSEARCH, as presented in **Observation 1**.

Observation 1. *When performing BAYESIANKEYSEARCH on a right ciphertext structure, the Wrong Key Response Profile helps Step 16 – 19 in Algorithm 1 to generate new candidates with low Hamming weight differences from the real subkey. These new candidates with a low Hamming weight difference are more likely to obtain higher scores. In contrast, when performing BAYESIANKEYSEARCH on a wrong ciphertext structure, the recommended candidates in the current iteration do not contribute to improving the scores in subsequent iterations.*

Furthermore, we observe the effect of the right and the wrong ciphertext structures on the key-recovery attack, as described in **Observation 2**.

Observation 2. *Given n_{cts} ciphertext structures, if the last two subkeys are correctly guessed, there will be a right ciphertext structure RC_s reaching the VS stage. If there is a wrong ciphertext structure WC_s reaches the VS stage, then the guesses for the last two subkeys are incorrect.*

Based on **Observation 2**, the attacker's goal is to ensure that at least one RC_s reaches VS while preventing any WC_s from passing the cutoff c_2 .

5.2 Analyzing the Parameters ($Cand_1, Step_1, c_1$) of BKS_1

The relationship between the desired success probability with the value of c_1 . Given a ciphertext structure $C_{structure}$, there are $Cand_1 \times Step_1$ candidates for the last subkey k_{-1} and their corresponding scores. Let $p_{>c_1}^r$ be the probability that, when executing BKS_1 on a right ciphertext structure, there is at least one candidate with a score higher than the cutoff c_1 . Similarly, let $p_{>c_1}^w$ be the probability that when executing BKS_1 on a wrong ciphertext structure, there is at least one candidate with a score exceeding c_1 .

For the right ciphertext structures, an essential prerequisite for a successful key-recovery attack is that there exists at least one right ciphertext structure that passes the cutoff c_1 . The upper bound of the attack success probability can be calculated as

$$\sum_{i=1}^{n_{cts}} P_i \times \left(1 - (1 - p_{>c_1}^r)^i\right) / P_{RC_s},$$

where n_{cts} is the total number of ciphertext structures, P_i is the probability that there are i right ciphertext structures among n_{cts} structures⁵, and P_{RC_s} is the probability that there is at least one right structure in the n_{cts} ciphertext structures.

For wrong ciphertext structure, approximately $p_{>c_1}^w \times n_{cts}$ structures pass the cutoff c_1 and proceed to BKS_2 . Since performing BKS_2 on wrong structures does not contribute to recovering the last two subkeys, it is necessary to reduce the number of wrong structure instances that pass the cutoff c_1 .

Therefore, the main objectives when tuning c_1 are:

⁵ P_i is calculated by $P_i = \binom{n_{cts}}{i} (2^{-13})^i (1 - 2^{-13})^{n_{cts}-i}$, where $\binom{n}{m}$ is the binomial coefficient.

- Task 1 : Increase $p_{>c_1}^r$ to ensure right structures pass the cutoff.
- Task 2 : Decrease $p_{>c_1}^w$ to prevent wrong structures from passing the cutoff.

We aim to minimize $p_{>c_1}^w$ to optimize time complexity while ensuring that $p_{>c_1}^r$ is sufficient to achieve the desired success probability P_S . Since an RC_s may still be rejected at the next cutoff c_2 , $p_{>c_1}^r$ must satisfy $p_{>c_1}^r \geq x$, where x is the solution to $\sum_{i=1}^{n_{cts}} P_i \times \left(1 - (1 - x)^i\right) / P_{RC_s} = P_S$, representing the lower bound of $p_{>c_1}^r$ required to achieve the desired P_S . For example, if we desire a success probability of $P_S = 0.99$, then $p_{>c_1}^r$ must be at least 0.9691.

Observation 3. *Given fixed values of $Cand_1$ and $Step_1$, as the cutoff value c_1 increases, the probabilities $p_{>c_1}^r$ and $p_{>c_1}^w$ both decrease; Conversely, as c_1 decreases, $p_{>c_1}^r$ and $p_{>c_1}^w$ both increase.*

Algorithm 2 Precomputation on BKS_1

Input: Set of ciphertext structures: $SC_{structure} := \{C_{structure}^0, \dots, C_{structure}^{n-1}\}$
Set of real last subkeys: $K_{real} := \{k_{real}^0, \dots, k_{real}^{n-1}\}$
Parameters of BKS_1 : $Cand_1$ and $Step_1$

Output: List L containing the result of performing BKS_1

- 1: $L \leftarrow \{\}$
- 2: **for** $i \in \{0, 1, \dots, n - 1\}$ **do**
- 3: $[ck, score, \max_{score}] \leftarrow BKS_1(C_{structure}^i)$
 / Perform BKS_1 on each ciphertext structure, obtaining $Cand_1 \times Step_1$ candidate subkeys and their scores. Both ck and $score$ contains $Cand_1 \times Step_1$ elements, and $\max_{score}$ is the maximum value among score. */*
- 4: $dk \leftarrow \{\}$
- 5: **for** $j \in \{0, 1, \dots, Cand_1 \times Step_1 - 1\}$ **do**
- 6: $dk \leftarrow dk \parallel ck_j \oplus k_{real}^i$
 / Replace the candidates with their differences from the real subkey. */*
- 7: **end for**
- 8: $L \leftarrow L \parallel [dk, score, \max_{score}]$
- 9: **end for**
- 10: **Return** L

Considering that an increase in $p_{>c_1}^w$ negatively impacts the time complexity, given the desired success probability P_S , the value of $p_{>c_1}^r$ is determined by selecting the minimum value of $p_{>c_1}^r$ that still satisfies the Inequation 3. Considering the interplay between c_1 , $p_{>c_1}^r$, and $p_{>c_1}^w$, we aim to select the minimum value of $p_{>c_1}^r$. This approach maximizes c_1 and minimizes $p_{>c_1}^w$, thereby reducing the number of wrong ciphertext structures passing the cutoff while ensuring a desired success probability.

$$\sum_{i=1}^{n_{cts}} P_i \times \left(1 - (1 - p_{>c_1}^r)^i\right) / P_{RC_s} \geq P_S. \quad (3)$$

Calculate the value of c_1 by the value of $p_{>c_1}^r$. Given the value of $p_{>c_1}^r$ and specific values for $Cand_1$ and $Step_1$, we proceed as follows:

- Determine c_1 : Perform precomputation using BKS_1 with the right structures as the input of **Algorithm 2**. The cutoff value c_1 is set to be the $\lceil p_{>c_1}^r \times n \rceil$ -th largest value among all $\max_{score}$, where n is the number of right ciphertext structures used in the precomputation.

- Obtain $p_{>c_1}^w$: Use the output of **Algorithm 2** with the wrong structures as the input to determine $p_{>c_1}^w$.

Example 1. Given a desired attack success probability of $P_S = 0.99$, we calculate the minimum required probability $p_{>c_1}^r = 0.9691$ for a right ciphertext structure to pass the cutoff c_1 to achieve P_S . Using the parameters $(Cand_1, Step_1) = (4, 40)$ and a set of 2^{14} right ciphertext structures along with their corresponding 2^{14} real last subkeys as input to **Algorithm 2**, we proceed as follows: From the output of **Algorithm 2**, we collect all the maximum scores ($\max_{score}$) obtained from the 2^{14} right ciphertext structures. We sort the $\max_{score}$ values in ascending order. The cutoff c_1 is set to the score at position: $\lceil 2^{14} \times 0.9691 \rceil = 15,878$. The calculated cutoff value is $c_1 = 0.68$.

With $p_{>c_1}^r = 0.9691$, **Algorithm 2** using multiple pairs of $(Cand_1, Step_1)$ is performed to obtain various values of c_1 and $p_{>c_1}^w$. The results are shown in **Figure 6**. These experiments are conducted using 2^{14} RC_{ss} and 2^{14} WC_{ss} .

The results in **Figure 6** demonstrate that increasing the number of iterations in BKS_1 decreases the probability of wrong structures passing the cutoff c_1 while keeping $p_{>c_1}^r$ fixed. Additionally, the results highlight the importance of effectively allocating the values of $Cand_1$ and $Step_1$, even when their product remains the same. For example, consider four pairs of parameters: $(Cand_1, Step_1) = (4, 160), (8, 80), (16, 40)$, and $(32, 20)$. Despite all pairs having the same product ($Cand_1 \times Step_1$), the probability $p_{>c_1}^w$ varies among them. Specifically, $p_{>c_1}^w$ exceeds 0.4 when using $(4, 160)$ and $(32, 20)$, while it is lower than 0.35 when using $(8, 80)$ and $(16, 40)$. Without considering BKS_2 , it is preferable to use $(8, 80)$ or $(16, 40)$ over $(4, 160)$ or $(32, 20)$. Therefore, given a fixed value of $Cand_1 \times Step_1$, it is important to test different allocations of $Cand_1$ and $Step_1$ to optimize the time complexity.

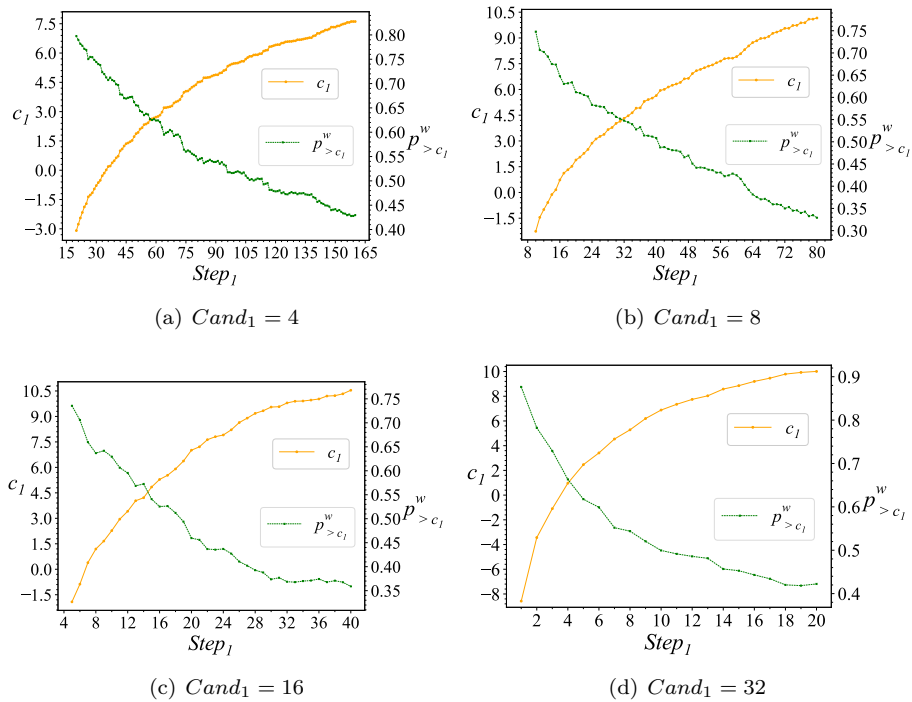


Figure 6: The curves of c_1 and $p_{>c_1}^w$ with $Cand_1 \in \{4, 8, 16, 32\}$, $p_{>c_1}^r = 0.9691$ and $Cand_1 \times Step_1 \leq 640$.

After determining the parameters $(Cand_1, Step_1, c_1)$, it is beneficial to understand what kind of candidate last subkeys can help a right structure pass the cutoff c_1 . This

question is addressed in [Observation 4](#), with further analysis provided in [Appendix C](#).

Observation 4. *When running BKS_1 on a right ciphertext structure, candidates whose difference from the real subkey has a higher mean response in the wrong key response profile are more likely to pass the cutoff c_1 .*

5.3 Analyzing the Parameters ($Cand_2, Step_2, c_2$) of BKS_2

The influence of the wrong structures on the choice of c_2 . In BKS_2 , the ciphertext structures that have passed the cutoff c_1 , along with several last subkey candidates whose scores exceed c_1 , are used to attempt the recovery of the second-to-last subkey. Following [Observation 2](#), the real subkey can be correctly guessed only when working with right structures. After completing the iteration in BKS_2 , the cutoff c_2 is applied to allow the right structure to proceed to the VS , while preventing wrong structures from advancing. To prioritize the attack's success probability, c_2 must be carefully selected to prevent any wrong structures among n_{cts} ciphertext structures from passing.⁶

As the bridge between the BKS_1 and the BKS_2 , the output of BKS_1 – which consists of multiple candidates for the last subkey with the scores exceeding c_1 and the associated ciphertext structure – is used as input to BKS_2 . For a wrong structure, the number of last subkey candidates from BKS_1 with scores higher than c_1 is denoted by $n_{>c_1}$, and the set of these candidates is denoted by $S_{>c_1}^w$. Using each member of $S_{>c_1}^w$, the wrong structure is decrypted by one round, and BKS_2 is performed once to generate $Cand_2 \times Step_2$ candidates for the second-to-last subkey and their corresponding scores.

Let $p_{>c_2}^w$ be the probability that, when performing BKS_2 on a wrong structure that is decrypted by one round using one candidate of the last subkey, there is at least one score exceeding the cutoff c_2 among all $Cand_2 \times Step_2$ scores. With that there are $n_{>c_1}$ candidates holding scores higher than c_1 in total, the probability that one wrong structure passes the second cutoff c_2 is $1 - (1 - p_{>c_2}^w)^{n_{>c_1}}$.

Without loss of generality, for each wrong structures, we assume that there are at most $n_{>c_1}$ candidates with the scores higher than c_1 . Actually, we limit the number of candidates that pass c_1 in the practical attacks to $n_{>c_1}$. There are at most $n_{>c_1}$ candidates used to decrypt the ciphertext structures for one round, whether the structure is right or wrong. Thus, given n wrong structures holding the scores higher than c_1 , the probability that at least one wrong structure passes the cutoff c_2 , denoted by P_w , is calculated by [Equation 4](#).

$$P_w = 1 - (1 - p_{>c_2}^w)^{n \times n_{>c_1}}. \quad (4)$$

Moreover, with using n_{cts} wrong structures in the practical attack, there are $n_{cts} \times p_{>c_1}^w$ wrong structures on average, holding the candidates with the scores higher than c_1 . As a result, when performing BKS_1 on each wrong structure and further performing BKS_2 on partial wrong structures, the probability that at least one wrong structure passing the cutoff c_2 is calculated by replacing n with $n_{cts} \times p_{>c_1}^w$ in [Equation 4](#).

Considering of [Observation 2](#), it is imperative that the probability P_w remains below an acceptable false positive rate, denoted by α . By setting $\alpha = 0.01$, we limit the likelihood of incorrect key recovery due to wrong structures to a negligible level. This requirement is mathematically expressed as [Inequation 5](#).

$$P_w = 1 - (1 - p_{>c_2}^w)^{n_{cts} \times p_{>c_1}^w \times n_{>c_1}} \leq \alpha. \quad (5)$$

Recalling [Equation 4](#) with replacing n with $n_{cts} \times p_{>c_1}^w$, the variable $p_{>c_1}^w$ is determined by the value of c_1 , and the n_{cts} and $n_{>c_1}$, being hyper-parameters, is explicitly defined prior

⁶Considering that there are only $2^{-13} \times n_{cts} RC_s$ instances among n_{cts} structures on average, the number of WC_s instances can be approximated by n_{cts} .

to the execution of key recovery operations. The only one variable related to the cutoff c_2 is $p_{>c_2}^w$. The subsequent analysis establishes an operational framework for calculating the value of c_2 by the upper bound of $p_{>c_2}^w$.

Calculating the value of c_2 by the the upper bound of $p_{>c_2}^w$. Before showing the analysis calculating the value of c_2 , the distribution of scores from performing the BKS_2 on the wrong structures is illustrated as follows.

When executing BKS_1 on wrong ciphertext structure, the recommended candidates in the current iteration fail to generate the new candidates with a low Hamming weight difference with the real subkey. In other words, this phenomenon indicates that the distribution of the differences between the high-scoring candidates (scores exceeding the cutoff c_1) with the real last subkeys is expected to approximately follow an uniform distribution. The advent of this phenomenon affords an high-efficiency method obtaining a lot of scores of performing BKS_2 on wrong structures, which is demonstrated in [Algorithm 3](#).

Algorithm 3 Precomputation on BKS_2 with Wrong Ciphertext Structures

Input: Set of wrong ciphertext structures: $SC_s := \{C_s^0, \dots, C_s^{n-1}\}$

Set of real last subkeys: $K_{real} := \{k_{real}^0, \dots, k_{real}^{n-1}\}$

Parameters of BKS_2 : $Cand_2$ and $Step_2$

Output: List L containing the result of performing BKS_2

```

1:  $L \leftarrow \{\}$ 
2: for  $i \in \{0, 1, \dots, n-1\}$  do
3:    $\delta \leftarrow$  randomly choose one value
4:    $\overline{C}_s^i \leftarrow Decrypt_{1r}(C_s^i, \delta \oplus k_{real}^i)$ 
      /* Decrypt  $C_s^i$  for one round using  $\delta \oplus k_{real}^i$  as the subkey. */
5:    $max_{score} \leftarrow BKS_2(\overline{C}_s^i)$ 
      /* Perform  $BKS_2$  on each ciphertext structure, obtaining  $Cand_2 \times Step_2$  candidate
      subkeys and their scores. And  $max_{score}$  is the maximum value among all scores. */
6: end for
7:  $L \leftarrow L \parallel [max_{score}]$ 
8: Return  $L$ 

```

Further, by viewing the maximal score as a random variable, the probability distribution of max_{score} can be estimated through a lot of output of [Algorithm 3](#), which is denoted by $\mathcal{F}$. Reviewing the definition of $p_{>c_2}^w$ – the probability that there is at least one score exceeding the cutoff c_2 when performing BKS_2 on a wrong structure decrypted by one round using one candidate, the bridge between the value of $p_{>c_2}^w$ and the value of c_2 is established by the cumulative distribution function of $\mathcal{F}$.

Manifestly, the probability – there is at least one score exceeding the cutoff c_2 when performing BKS_2 on a right structure – exhibits an inverse dependence on the value of c_2 . Thus, the c_2 should be minimized to maximize the probability of right structures passing the c_2 . Considering both minimizing c_2 and satisfying Inequation 5, the value of $p_{>c_2}^w$ is set to be the upper bound of $p_{>c_2}^w$. Further, the value of c_2 is determined by the value of $p_{>c_2}^w$ and the cumulative distribution function of $\mathcal{F}$.

Based on the above analysis about calculating c_2 , the formal implementation is presented as follows.

- 1) Given $(Cand_1, Step_1)$, the value of c_1 is determined by [Section 5.2](#). And the value of $p_{>c_1}^w$ is calculated.
- 2) Given $(Cand_2, Step_2)$, the probability distribution $\mathcal{F}$ can be estimated by the output of [Algorithm 3](#).

- 3) Calculate the value of $p_{>c_2}^w$ satisfying $1 - (1 - p_{>c_2}^w)^{n_{cts} \times p_{>c_1}^w \times n_{>c_1}} = \alpha$, where $\alpha = 0.01$, the values of n_{cts} and $n_{>c_1}$ are predetermined.
- 4) With the cumulative distribution function of $\mathcal{F}$, the value of c_2 is determined.

For clarity, we recall the definitions of above parameters shown in Table 3. Due to the artificially imposed limitation on the wrong ciphertext structures (e.g., a maximum of $n_{>c_1}$ as mentioned earlier) – which restricts the number of times performing BKS_2 on each wrong structure to at most $n_{>c_1}$ – we inevitably impose the same constraint on each right structure. This means that BKS_2 is performed no more than $n_{>c_1}$ times for per right structure after each call of BKS_1 . If there are more than $n_{>c_1}$ candidate last subkeys with scores exceeding c_1 , only the top $n_{>c_1}$ candidates with the highest scores are used in BKS_2 to attempt recovery of the second-to-last subkey.

Table 3: The parameters used to calculate the value of c_2 .

Notation	Definition	Value Derivation
$p_{>c_1}^w$	The probability that there is at least one candidate with a score higher than c_1 when executing BKS_1 on one wrong structure.	See Section 5.2
$\mathcal{F}$	The probability distribution of the output of Algorithm 3.	Simulate the statistical distribution using the output of Algorithm 3
n_{cts}	The number of wrong structures.	Predetermined
$n_{>c_1}$	Executing BKS_2 at most $n_{>c_1}$ times for each ciphertext structure.	Predetermined
$p_{>c_2}^w$	The probability that there is at least one score exceeding the cutoff c_2 when performing BKS_2 on a wrong structure decrypted by one round using one candidate.	Satisfy $1 - (1 - p_{>c_2}^w)^{n_{cts} \times p_{>c_1}^w \times n_{>c_1}} = \alpha$

Introducing the new parameter $n_{>c_1}$ raises a new question: How should we choose the optimal value of $n_{>c_1}$ to maximize the probability that a right ciphertext structure passes the cutoff c_2 ? To address this, we propose a new algorithm in Section 5.4 that determines the best choice of $n_{>c_1}$.

5.4 Parameter Tuning

In Section 5.3, we introduce a new variable $n_{>c_1}$ for computing the value of c_2 . Through theoretical analysis, we establish that both parameters exhibit coupled effects on the probability of right ciphertext structures passing the two cutoffs. The probability that the right structure passing the two cutoffs is denoted by P_{BKS_1, BKS_2}^r . Specifically, the derived c_2 directly governs P_{BKS_1, BKS_2}^r , while $n_{>c_1}$ indirectly modulates this probability through its relationship with c_2 .

Since the parameter space of $n_{>c_1}$ spans from 1 to $Cand_1 \times Step_1$, we implement a brute-force search to identify the optimal choice of $n_{>c_1}$ that can maximize the probability P_{BKS_1, BKS_2}^r . However, for each candidate value of $n_{>c_1}$, it requires determination of the corresponding value of c_2 followed by experimentally obtaining the probability that the right structure passes the two cutoffs, where it is time-consuming when calculating P_{BKS_1, BKS_2}^r . For this, we introduce an efficient algorithm calculating the value of P_{BKS_1, BKS_2}^r by reusing the output of Algorithm 2.

Calculating the value of P_{BKS_1, BKS_2}^r . Given the specific values of the seven parameters ($Cand_1, Step_1, c_1, Cand_2, Step_2, c_2, n_{>c_1}$), P_{BKS_1, BKS_2}^r is calculating by online performing BKS_1 and BKS_2 on a large number of right structures. However, it is time-intensive

if given multiple groups of values. For this, we aim at directly estimating the value of P_{BKS_1, BKS_2}^r without executing BKS_1 .

With **Observation 4** and **Appendix C**, the high-quality candidates whose differences with the real subkeys have a higher mean response in $WCRP$ are more likely to hold the higher scores and pass the first cutoff c_1 , when executing BKS_1 on the right structures. Besides, when performing BKS_2 on the right structures decrypted one round using the candidates, it is found that the output of BKS_2 are likely to hold the higher scores if the candidates of last subkeys hold the low-Hamming weight differences with the real subkeys. Thus, we can establish a connection between BKS_1 and BKS_2 through the high-quality candidates whose differences with the real subkeys exhibit two critical constraints: (a) have a higher mean response in $WCRP$; (b) are low-Hamming weight. The difference between the real last subkey and the high-quality candidate is defined as the high-quality difference. The set of the high-quality differences is denoted Diff_{hq} .

Algorithm 4 Precomputation on BKS_2 with Right Ciphertext Structures

Input: Set of right ciphertext structures: $SC_s := \{C_s^0, \dots, C_s^{n-1}\}$

Set of real last subkeys: $K_{real} := \{k_{real}^0, \dots, k_{real}^{n-1}\}$

Parameters of BKS_2 : $Cand_2$ and $Step_2$

Set of Differences: $\text{Diff} := \{\text{diff}_0, \dots, \text{diff}_1\}$

Output: List L containing the result of performing BKS_2

```

1:  $L \leftarrow \{\}$ 
2: for  $i \in \{0, 1, \dots, n-1\}$  do
3:    $\delta \leftarrow$  randomly choose one value from  $\text{Diff}$ 
4:    $\overline{C}_s^i \leftarrow \text{Decrypt}_{1r}(C_s^i, \delta \oplus k_{real}^i)$ 
      /* Decrypt  $C_s^i$  for one round using  $\delta \oplus k_{real}^i$  as the subkey. */
5:    $\max_{score} \leftarrow BKS_2(\overline{C}_s^i)$ 
      /* Perform  $BKS_2$  on each ciphertext structure, obtaining  $Cand_2 \times Step_2$  candidate
      subkeys and their scores. And  $\max_{score}$  is the maximum value among all scores. */
6: end for
7:  $L \leftarrow L \parallel [\max_{score}]$ 
8: Return  $L$ 

```

The probability that the right ciphertext structure holding the scores higher than c_1 passes the second cutoff c_2 is calculated as follows.

- **Scenario A.** For the high-quality difference Δ , performing **Algorithm 4** with N right structures and $\text{Diff} := \{\Delta\}$ results in n output values exceeding c_2 among all outputs. Thus, given one candidate with that the difference with the real last subkey is Δ , the probability that the right structure decrypted one round using the candidate passes the second cutoff, denoted by $P_{\Delta > c_2}$, is $\frac{n}{N}$. Further, supposing that the right structure is allowed to pass the c_2 only when the candidate that the difference with the real last subkey is Δ , the probability of one right structure passing the BKS_2 is $P_{\Delta > c_2}$.
- **Scenario B.** Actually, for most of right structure, there are multiple high-quality differences making the probability of one right structure passing the BKS_1 and BKS_2 not zero. Given one right structure passing the cutoff c_1 , there are t candidates with scores exceeding c_1 among top $n_{>c_1}$ candidates with the highest scores. As a result, the probability of the right structures passing the c_2 is $1 - \prod_{i=0}^{t-1} (1 - P_{\Delta_i > c_2})$, where Δ_i is the difference between i -th candidate with the real last subkey.

Since there are numerous high-quality differences, it is time-consuming performing **Algorithm 4** for each differences in **Scenario B**. For this, we partition high-quality difference

set Diff_{hq} into multiple subsets (*i.e.*, $\{\text{Diff}_0, \text{Diff}_1, \dots, \text{Diff}_m\}$) to optimize computational efficiency, where $\text{Diff}_{hq} = \bigcup_{i=0}^m \text{Diff}_i$ and $\text{Diff}_i \cap \text{Diff}_j = \emptyset$ when $i \neq j$.

- **Scenario C.** Given one difference subset $\{\Delta_0, \Delta_1, \dots, \Delta_m\}$, performing [Algorithm 4](#) with N right structures and $\text{Diff} := \{\Delta_0, \Delta_1, \dots, \Delta_m\}$ results in n output values exceeding c_2 among all output. With one candidate holding that the difference with the real last subkey belongs to $\{\Delta_0, \Delta_1, \dots, \Delta_m\}$, the probability that the right structure decrypted one round using the candidate passes the second cutoff, denoted by $P_{\text{Diff} > c_2}$, is $\frac{n}{N}$.
- **Scenario D.** With that there are t candidates with scores exceeding c_1 among top $n_{>c_1}$ candidates with the highest scores, these differences, between these candidates with the real subkey, belong to multiple difference sets. Thus, the probability of the right structure passing c_2 is $1 - \prod_{i=0}^m (1 - P_{\text{Diff}_i > c_2})^{n_i}$, where the n_i denotes that there are n_i differences belonging to Diff_i , $\sum_{i=0}^m n_i = t$.

In **Scenario D**, we set the probability of different differences in the one difference set to the same value. Such approximations inevitably trade computational efficiency for diminished accuracy in estimating the probability of the right structure passing the cutoff c_2 . Moreover, in **Scenario D**, we only execute [Algorithm 3](#) on the high-quality differences. And, when decrypted by the low-quality candidate, the probability of the right structure passing c_2 is fixed to 0, where the candidate is low-quality if its difference with the real subkey is not in Diff_{hq} . With the above analysis, given the specific values of the seven parameters $(\text{Cand}_1, \text{Step}_1, c_1, \text{Cand}_2, \text{Step}_2, c_2, n_{>c_1})$, the probability of the right structures passing the two cutoffs c_1 and c_2 is calculated by [Algorithm 5](#).

In [Algorithm 5](#), the high-quality difference set is divided into m subsets. Given the output of performing [Algorithm 2](#) on N right structures, the probability of passing the c_1 and c_2 is calculated in Step 5-Step 20. In Step 18, the probability of the i -th right structure passing the two cutoffs is $\overline{P_{BKS_2}}$ (see **Scenario D**). For the right structures without scores exceeding c_1 , the probability of passing the c_1 and c_2 is 0. For the right structures with scores higher than c_1 , the probability that there are at least one candidate of the second-to-last subkey holding the score exceeding c_2 , when performing BKS_2 on the right structure, is calculated by evaluating each candidates of the last subkey with the high-quality difference and score exceeding c_1 .

Utilizing [Algorithm 5](#), the connection between $(c_2, n_{>c_1})$ and P_{BKS_1, BKS_2}^r is established. Based on this, we can choose the best choices of $(c_2, n_{>c_1})$ by maximizing P_{BKS_1, BKS_2}^r .

Best choices of $(c_2, n_{>c_1})$. Given the specific values of $(\text{Cand}_1, \text{Step}_1, \text{Cand}_2, \text{Step}_2)$, the value of c_1 is determined by the probability of the right structures passing c_1 , shown in [Section 5.2](#). The value of c_2 is related to the value of $n_{>c_1}$. However, the value of $n_{>c_1}$ remains unknown. To address this requirement, we develop an optimization framework that searches the best choice of $n_{>c_1}$, illustrated in [Algorithm 6](#).

In [Algorithm 6](#), the all candidates of $n_{>c_1}$ are exhaustively enumerated until the maximum value of P_{BKS_1, BKS_2}^r is obtained. In addition to obtaining the best choices of $n_{>c_1}$ and c_2 , P_{BKS_1, BKS_2}^r —the output of [Algorithm 6](#)—is used further to estimate the success rate of the key-recovery attacks.

Given P_{BKS_1, BKS_2}^r and n_{cts} ciphertext structures, the probability that at least one right ciphertext structure reaches the VERIFIERSEARCH (VS) stage, denoted by P_{VS} , is calculated using [Equation 6](#).

$$P_{VS} = \sum_{i=1}^{n_{cts}} P_i \times \left(1 - (1 - P_{BKS_1, BKS_2}^r)^i \right) / P_{RC_s}. \quad (6)$$

Algorithm 5 Calculating the value of P_{BKS_1, BKS_2}^r

Input: Set of High-Quality Differences: $\text{Diff}_{hq} = \bigcup_{i=0}^{m-1} \text{Diff}_i$
 L : The output of performing **Algorithm 2** using N right structures
Hyper-Parameters: $(\text{Cand}_1, \text{Step}_1, c_1, \text{Cand}_2, \text{Step}_2, c_2, n_{>c_1})$

Output: P_{BKS_1, BKS_2}^r

```

1:  $n_{BKS_1, BKS_2}^r \leftarrow 0$ 
2: for  $i \in \{0, 1, \dots, m-1\}$  do
3:   Perform Algorithm 4 to calculate the value of  $P_{\text{Diff}_i > c_2}$ 
4: end for
5: for  $i \in \{0, 1, \dots, N-1\}$  do
6:    $\frac{P_{BKS_2}}{P_{BKS_2}} \leftarrow 1$ 
7:    $\frac{P_{BKS_2}}{P_{BKS_2}} \leftarrow 0$ 
8:   if  $L[i].\text{max}_{\text{score}} > c_1$  then
9:      $D, \text{Score}_D \leftarrow \text{sort}_{L[i].dk}(L[i].\text{score})[-n_{>c_1} :]$ 
10:    /* Pick  $n_{>c_1}$  highest scores and the corresponding differences. */
11:    for  $j \in \{0, 1, \dots, n_{>c_1}-1\}$  do
12:       $p \leftarrow 0$ 
13:      if  $\text{Score}_D[j] > c_1$  and  $D[j] \in \text{Diff}_{hq}$  then
14:         $p \leftarrow P_{\text{Diff}_t > c_2}$  if  $D[j] \in \text{Diff}_t$ 
15:      end if
16:       $P_{BKS_2} \leftarrow P_{BKS_2} \times (1 - p)$ 
17:    end for
18:     $\frac{P_{BKS_2}}{P_{BKS_2}} \leftarrow 1 - P_{BKS_2}$ 
19:     $n_{BKS_1, BKS_2}^r \leftarrow n_{BKS_1, BKS_2}^r + \frac{P_{BKS_2}}{P_{BKS_2}}$ 
20:  end for
21:  $P_{BKS_1, BKS_2}^r \leftarrow n_{BKS_1, BKS_2}^r / N$ 
22: Return  $P_{BKS_1, BKS_2}^r$ 

```

Algorithm 6 An optimization framework that selects $n_{>c_1}$ and c_2

Input: Hyper-Parameters: $(\text{Cand}_1, \text{Step}_1, c_1, \text{Cand}_2, \text{Step}_2)$

Output: $\overline{n}_{>c_1}, \overline{c}_2, \overline{P}_{BKS_1, BKS_2}^r$

```

1:  $\overline{n}_{>c_1} \leftarrow 0$ 
2:  $\overline{c}_2 \leftarrow 0$ 
3:  $\overline{P}_{BKS_1, BKS_2}^r \leftarrow 0$ 
4: for  $n_{>c_1} \in \{1, 2, \dots, \text{Cand}_1 \times \text{Step}_1\}$  do
5:   Calculate the value of  $c_2$  following the analysis of Section 5.3
6:    $P_{BKS_1, BKS_2}^r \leftarrow$  the output of performing Algorithm 5 using the seven parameters
    $(\text{Cand}_1, \text{Step}_1, c_1, \text{Cand}_2, \text{Step}_2, c_2, n_{>c_1})$ 
7:   if  $P_{BKS_1, BKS_2}^r > \overline{P}_{BKS_1, BKS_2}^r$  then
8:      $\overline{n}_{>c_1} \leftarrow n_{>c_1}$ 
9:      $\overline{c}_2 \leftarrow c_2$ 
10:     $\overline{P}_{BKS_1, BKS_2}^r \leftarrow P_{BKS_1, BKS_2}^r$ 
11:   end if
12: end for
13: Return  $\overline{n}_{>c_1}, \overline{c}_2, \overline{P}_{BKS_1, BKS_2}^r$ 

```

where P_i is the probability that there are i right ciphertext structures RC_s s among the n_{cts} structures, and P_{RC_s} is the probability that there is at least one right structure among the n_{cts} structures. Due to that the last two subkeys are guessed correctly if and only if there is at least one right structure passing c_1 and c_2 , the success rate can be estimated by the value of P_{VS} .

5.5 Evaluation of parameter optimization strategy

Given $(Cand_1, Step_1, Cand_2, Step_2)$, the c_1 is determined in Section 5.2. And the value of c_2 is determined following the analysis in Section 5.3 and Section 5.4. Besides that, the probability of the right structures visiting VS is given. With that the best choice of c_2 is recommend mainly by maximizing P_{BKS_1, BKS_2}^r , it is essential to measure the difference between the theoretical value of P_{BKS_1, BKS_2}^r with its experimental value.

Taking selecting parameters for the 18-round related-key neural-differential attacks on SIMON32/64 as an example, we apply the parameter optimization strategy on various groups of $(Cand_1, Step_1, Cand_2, Step_2)$. Let S_{num} be the set of differences to the real subkey that has the highest num mean responses in $WCRP$ (see Appendix C). Employing S_{num} as the high-quality difference set $Diff_{hq}$, the S_{num} is used as the input of Algorithm 5 and we don't partition S_{num} into multiple subsets. We obtain the multiple values of P_{BKS_1, BKS_2}^r by employing S_{16} , S_{32} , S_{64} , S_{128} and S_{256} and choose the maximum value among these values as the theoretical value of P_{BKS_1, BKS_2}^r . The recommend cutoffs is shown in Table 4.

Table 4: The recommended choices given the parameters of BKS_1 and BKS_2 with $(Cand_2 = 8, Step_2 = 10)$. The cutoff c_1 is recommended by Section 5.2 and the c_2 is recommended by Section 5.3 and Section 5.4.

$(Cand_1, Step_1)$	c_1	c_2	$n_{>c_1}$	P_{BKS_1, BKS_2}^r (by Algorithm 6)	P_{BKS_1, BKS_2}^r (by experiment)
(4, 20)	-2.71	-588	19	0.321 with S_{128}	0.324
(4, 40)	0.68	-589	19	0.535 with S_{128}	0.533
(4, 80)	4.26	-591	20	0.712 with S_{128}	0.728
(4, 160)	7.88	-594	20	0.824 with S_{128}	0.857
(8, 10)	-2.53	-588	19	0.372 with S_{128}	0.377
(8, 20)	1.47	-589	20	0.578 with S_{128}	0.601
(8, 40)	5.67	-592	20	0.725 with S_{128}	0.736
(8, 80)	10.21	-597	20	0.803 with S_{128}	0.808
(16, 5)	-2.07	-594	11	0.351 with S_{64}	0.397
(16, 10)	2.30	-590	20	0.578 with S_{128}	0.608
(16, 20)	6.41	-592	20	0.698 with S_{128}	0.720
(16, 40)	10.47	-595	20	0.757 with S_{128}	0.741
(32, 2)	-3.37	-608	3	0.112 with S_{64}	0.121
(32, 5)	2.79	-596	12	0.570 with S_{64}	0.606
(32, 10)	7.02	-592	20	0.697 with S_{128}	0.719
(32, 20)	10.26	-594	20	0.744 with S_{128}	0.743

Given the recommended cutoffs, we execute BKS_1 on N right structures and further perform BKS_2 on the right structures with scores higher than c_1 . Among these N right structures, there are n structures allowed to access to VS . The experimental value of P_{BKS_1, BKS_2}^r is $\frac{n}{N}$, which is shown in the last column of Table 4. As shown in the last two columns of Table 4, the difference between the theoretical value and the experimental value is negligible, demonstrating the effectiveness of the cutoff selection strategy. In addition,

the results indicate that the probability of a right structure passing the cutoffs c_1 and c_2 increases with the number of iterations.

6 Key Recovery Attack on Round-Reduced SIMON32/64

6.1 Improved Single-Key Recovery Attack on 16-Round SIMON32/64

We enhance the 16-round key-recovery attack on SIMON32/64 proposed by Bao *et al.* [BGL⁺21, BGL⁺22] by applying our parameter optimization strategy (Section 5). This leads to new parameters and improved attack complexity.

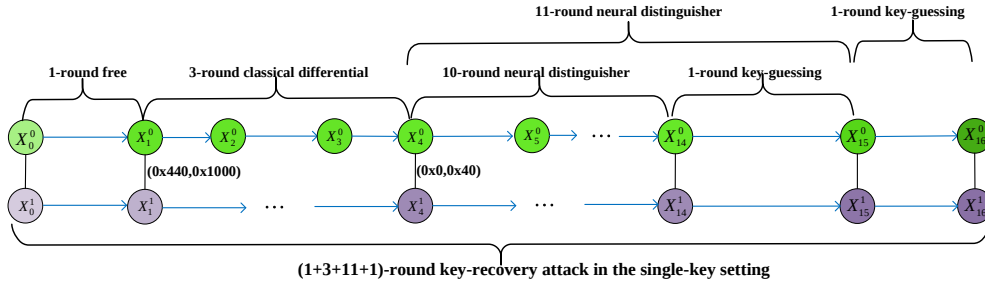


Figure 7: Components of the key-recovery attacks of 16-round SIMON32/64

6.1.1 Brief Description of Initial Parameters

Using the 3-round differential $(0x440, 0x1000) \rightarrow (0x0, 0x40)$ with probability approximately $2^{-7.3}$, the attack in [BGL⁺21, BGL⁺22] uses 11 neutral bits to generate ciphertext structures: $\{[2], [3], [4], [6], [8], [9], [10], [18], [22], [0, 24], [12, 26]\}$. The attack parameters are $(Cand_1 = 32, Step_1 = 5, c_1 = 25, Cand_2 = 32, Step_2 = 5, c_2 = 100)$, utilizing 2^9 ciphertext structures to recover the last two subkeys. The BKS_1 algorithm executes up to $n_{it} = 2^{11}$ times, with structures selected based on a priority formula from [Goh19, BGL⁺22].

For the 3-round classical distinguisher, it is found that (X_1^0, X_1^1) is the conforming pair of the 3-round differential then $X_1^0[17] = X_1^1[17] = 0, X_1^0[19] = X_1^1[19] = 0$. Thus, Bao *et al.* firstly guess the two bits of the first subkey, $k_0[0]$ and $k_0[3]$ and choose the ciphertext structures holding $X_1^0[17] = X_1^1[17] = 0, X_1^0[19] = X_1^1[19] = 0$. Actually, the set of structures can be divided into four subsets following the different values of $k_0[0]$ and $k_0[3]$, where each subset contains 2^7 structures. The times of performing BKS_1 on each subset is no more than 2^9 . Running the attacks on the subset containing the right structures, 49 successful trials with 99 trails in total were conducted, for which the returned last two subkeys have a Hamming distance to the real subkeys of at most two.

Moreover, a practical rate of the right structures passing the cutoffs c_1 and c_2 is given using the initial parameters. Using 2^{14} right structures and $(Cand_1 = 32, Step_1 = 5, c_1 = 25, Cand_2 = 32, Step_2 = 5, c_2 = 100)$, $P_{BKS_1, BKS_2}^r = 0.032$.

6.1.2 Multiple Groups of the New Parameters

To improve success probability and reduce attack complexity, we adjust the cutoffs c_1 and c_2 using our optimization strategy, considering different probabilities $p_{>c_1}^r$ of right ciphertext structures passing c_1 . The recommended cutoffs and corresponding probabilities are shown in Table 5.

To further reduce time complexity, we implement two methods: (1) limiting the maximal number of iterations to $n_{it} = 2^9 + 2^8$, and (2) reducing the number of candidates

Table 5: Recommended cutoffs with $(Cand_1, Step_1, Cand_2, Step_2) = (32, 5, 32, 5)$ and $n_{it} = 2^{11}$.

$p_{>c_1}^r$	c_1	c_2	$n_{>c_1}$	P_{BKS_1, BKS_2}^r	$p_{>c_1}^w$
0.2	18.70	19	23	0.1699 with S_{256}	0.0702
0.3	16.28	22	29	0.2381 with S_{256}	0.1423
0.4	14.11	24	34	0.2941 with S_{256}	0.2358
0.5	12.10	26	47	0.3397 with S_{256}	0.3390

recommended by BKS_2 by setting $Cand_2 \times Step_2 = 80$ instead of 160. The recommended cutoffs with these adjustments are shown in Table 6.

Table 6: Recommended cutoffs with $(Cand_1 = 32, Step_1 = 5, Cand_2 = 8, Step_2 = 10)$ and $n_{it} = 2^9 + 2^8$. exp.: experimental result

$p_{>c_1}^r$	c_1	c_2	$n_{>c_1}$	P_{BKS_1, BKS_2}^r	P_{BKS_1, BKS_2}^r (exp.)	$p_{>c_1}^w$	Success rate	Time ($\times T_{base}$)
0.2	18.70	24	19	0.1663 with S_{256}	0.1669	0.0702	52/99	$2^{-1.20}$
0.3	16.28	29	19	0.2284 with S_{256}	0.2285	0.1423	63/99	$2^{-0.79}$
0.4	14.11	33	20	0.2803 with S_{256}	0.2949	0.2358	67/99	$2^{-0.40}$

6.1.3 Key-Recovery Attack

Similar to the attacks in [BGL⁺22], all attack trails are run on the subset holding $X_1^0[17] = X_1^1[17] = 0$, $X_1^0[19] = X_1^1[19] = 0$, and each trial has the right structures. Given $X_1^0[17] = X_1^1[17] = 0$, $X_1^0[19] = X_1^1[19] = 0$, the probability that (X_1^0, X_1^1) is the conforming pair of the 3-round differential is about $2^{-5.3}$. The probability of that there are i right structures in 2^7 structures can be obtained. The probability that there is at least one right structure is $P_{RC_{structure}} = 1 - (1 - 2^{-5.3})^{2^7}$.

Generating Ciphertext Structures Used in Key-Recovery Attacks. With the 3-round classical differential distinguisher $(0x440, 0x1000) \rightarrow (0x0, 0x40)$ used in the 16-round attack, the ciphertext structures are generated using 11 neutral bit sets $I_s = \{[2], [3], [4], [6], [8], [9], [10], [18], [22], [0, 24], [12, 26]\}$. The n_{cts} ciphertext structures are constructed as follows:

1. Randomly generate n_{cts} plaintexts, i.e., $\tilde{P}$.
2. Modify the [17] and [19] bit by $\tilde{P} = \tilde{P} \odot 0xfff5ffff$.
3. Generate the data pairs, i.e., $(\tilde{P}, \tilde{P}')$, with difference $(0x440, 0x1000)$.
4. Extend each data pair to one structure using I_s .
5. Decrypt one round using zero as the subkey for all data in the structures obtained above and obtain n_{cts} plaintext structures.
6. Query for the ciphertexts under 16-round SIMON32/64, obtaining n_{cts} ciphertext structures.

Success Rate. The theoretical success rates, considering the probabilities of right structures accessing the VERIFIERSEARCH (VS) stage, are calculated based on our parameters (see Equation 6). For the parameters shown in Table 6, when performing BKS_1 only once for each structure, the probabilities of the right structures visiting VS are 0.4356,

0.5453, 0.6417, respectively. Due to $n_{it} > n_{cts}$, it is likely that the right structure is chosen again in the last $n_{it} - n_{cts}$ iterations. Thus, the theoretical success rates are more than 0.4356, 0.5453, 0.6417.

For comparison, we conducted 99 trials of the attack core, matching the number performed in [BGL⁺22]. We count a key-recovery attack as successful if the right structure is used as the input of VS . Under these conditions, the attacks are successful in 52, 63, 67 out of 99 trails, respectively.

Time Complexity. We use the time complexity shown in [BGL⁺22] as the baseline instead of directly calculating the time complexity. For the attack in [BGL⁺22], the average running time of the wrong structure in one iteration is denoted by t_{base} ⁷. With that there are at most 2^{11} iterations, the time complexity in the worst case is $2^{11} \times t_{base}$, denoted by T_{base} . For the attacks using the parameters shown in Table 6, the average running time of the wrong structure in one iteration is denoted by t_{new} . With that there are at most $2^9 + 2^8$ iterations, the time complexity in the worst case is $(2^9 + 2^8) \times t_{new}$. Thus, the time complexity using the new parameters is $\frac{2^9 + 2^8}{2^{11}} \times \frac{t_{new}}{t_{base}} \times T_{base}$. It is noticed that the values of t_{base} and t_{new} need to be calculated on the same software and hardware equipment.

6.2 Related-Key Recovery Attack on 18-Round SIMON32/64

6.2.1 The Parameters for Key Recovery Attack

Balancing success rate and time complexity, we investigated multiple parameters of BKS_1 and BKS_2 . As shown in Table 4, with $(Cand_2 = 8, Step_2 = 10)$ and $Cand_1 \times Step_1 = 160$, multiple choices of $(Cand_1, Step_1)$ yield similar P_{BKS_1, BKS_2}^r , which influences the success rate. However, there is an obvious difference in $p_{>c_1}^w$, the probability that wrong structure passing the cutoff c_1 . Higher values of $p_{>c_1}^w$ increase unnecessary computations on wrong structures without aiding key recovery. Comparing $(Cand_1, Step_1) = (4, 40), (8, 20), (16, 10), (32, 5)$, with corresponding $p_{>c_1}^w$ values of 0.685, 0.652, 0.620, and 0.589, we chose (32, 5) for BKS_1 due to its lower value of $p_{>c_1}^w$. The selected parameters are summarized in Table 7.

Table 7: Parameters for the 18-Round SIMON32/64.

Parameter	Value	Parameter	Value	Parameter	Value	Parameter	Value
$Cand_1$	32	$Step_1$	5	c_1	2.79	$n_{>c_1}$	12
$Cand_2$	8	$Step_2$	10	c_2	-596	n_{cts}	2^{14}

6.2.2 Details of Key-Recovery Attack

Generating Ciphertext Structures Used in Key-Recovery Attacks. With the prepended 3-round differential distinguishers in the single-key and related-key settings, the ciphertext structure is constructed as follows:

1. Randomly generate X_1^0 . Obtain the ordered quadruple $(X_1^0, X_1^1, X_1^2, X_1^3)$ using: $X_1^1 = X_1^0 \oplus (0x440, 0x1000)$, $X_1^2 = X_1^0 \oplus (0x1d0, 0x754)$, $X_1^3 = X_1^0 \oplus (0x440, 0x1000) \oplus (0x1d0, 0x754)$.
2. Expand the ordered quadruple into a structure S_{middle} containing 2^{13} ordered

⁷With randomly generating 2^{12} wrong structures, $t_{base} = 3.99s$ conducted by Python 3.8.16 and Tensorflow 2.5.0. The device information is Intel Core i5-13600K with 5.10GHz and NVIDIA RTX4080 SUPER.

quadruples by applying 13 neutral bits⁸. Denote the set of neutral bits is denoted by I_s . Let $\mathcal{S}$ denote the subspace of $\mathbb{F}_2^{32}$ with basis $\{e_i | i \in I_s\}$, where e_i is the vector with a 1 in the i -th position and zeros elsewhere. Each element in S_{middle} has the form of $(X_1^0 \oplus \psi, X_1^1 \oplus \psi, X_1^2 \oplus \psi, X_1^3 \oplus \psi)$, where $\psi \in \mathcal{S}$.

3. Decrypt the middle state structure from Step 2 by one round to obtain the plaintext structure, using 0 and 0x4 as the subkeys. Specifically, the first two elements are decrypted using 0 as the subkey, and the last two elements are decrypted using 0x4 as the subkey.⁹
4. Encrypt the plaintext structure for 18 rounds using K and $K \oplus \Delta_{RK}$ as the master keys. The first four subkey differences between K and $K \oplus \Delta_{RK}$ are $(\Delta_{k_0}, \Delta_{k_1}, \Delta_{k_2}, \Delta_{k_3}) = (0x4, 0x74, 0x50, 0x60)$. Similarly, the first two elements in each quadruple of the plaintext structure are encrypted using K as the master key, and the last two elements are encrypted using $K \oplus \Delta_{RK}$.

Attack Procedure. Using the above method, we employed 2^{14} ciphertext structures and ran BKS_1 on each. Structures scoring above $c_1 = 2.79$ in BKS_1 proceeded to BKS_2 . Those scoring above $c_2 = -596$ in BKS_2 accessed the VERIFIERSEARCH (VS) stage, which returned guesses for the last two subkeys $k_{17}||k_{16}$. The attack procedure is as follows.

1. Initialize an array w_{max} and an array n_{visit} to record the highest scores and the numbers of visits obtained by ciphertext structures.
2. Choose one ciphertext structure from the set of ciphertext structures one by one and the index of the current structure is denoted by idx ;
3. Run the BKS_1 on the chosen structure. Update the value of $w_{max}[idx]$ by the maximum value of $Cand_1 \times Step_1$ scores. And $n_{visit}[idx] \leftarrow n_{visit}[idx] + 1$. If there is at least one candidates with score over c_1 , go to Step 4; else go to Step 2 to choose the next ciphertext structure;
4. Decrypt the structure one round using candidates with a score greater than c_1 to generate multiple structures decrypted one round. Run the BKS_2 on these structures decrypted one round. If there is at least one candidates with score over c_2 , go to Step 5; else go to Step 2 to choose the next ciphertext structure;
5. Given the $n_{>c_1} \times Cand_2 \times Step_2$ scores, find the maximum score and its corresponding candidates of the last subkey and the second to last subkey. Run the VS using the two candidates and the current structure. And the key-recovery attack is terminated after running the VS .

Based on our analysis in Section 5, the structure that accesses the VS stage is expected to be the right ciphertext structure. Once this ciphertext structure is identified, we can recover 7 bits of the first subkey k_0 at positions $[0, 1, 2, 11, 13, 14, 15]$, since the right structure satisfies Equation 2. Specifically, $k_0[i] = X_0^0[i]$ for $i \in \{0, 1, 2, 11, 13, 14, 15\}$, where X_0^0 is the first plaintext of the initial quadruple in the right structure.

With the known values of k_{17} and k_{16} , we generated 2^{32} candidates for k_{15} and k_{14} . Using the linear key schedule of SIMON32/64, we derived k_0 , k_1 , and k_2 . Given the 7 known bits of k_0 , the number of candidates reduced to 2^{25} on average. Applying Equation 2, we further reduced the candidates to approximately 2^{17} possibilities.

⁸The accuracy of the combine-response distinguisher of $\mathcal{RK}\text{-}\mathcal{ND}^{\text{SIMON13R}}$ exceeds 99.9% when using 2^{13} samples from the same distribution. The 13 deterministic neutral bits are $I_s = \{[0], [2], [3], [4], [5], [6], [7], [8], [9], [10], [18], [20], [22]\}$.

⁹For the round function of SIMON32/64, when (P, P') is decrypted one round and then encrypted one round to $(\bar{P}, \bar{P}')$ using different keys, we have $P \oplus P' = \bar{P} \oplus \bar{P}'$.

Success Rate. When at least one right structure exists, the probability of a wrong structure accessing VS is negligible – at most once in 100 experiments on average. In the remaining 99 experiments, approximately 99×0.7864 have the right structure allowed to access VS , yielding a theoretical success rate of about 77.85%. In our experiment, out of 100 experiments, the last two subkeys were rightly guessed in 82 cases, closely aligning with the theoretical prediction. Considering the probability of having at least one right structure in 2^{14} structures is 0.86, the overall success rate is approximately 65.9%.

6.2.3 Complexity Analysis of the Key Recovery Attack

- 1) Data complexity. Utilizing 2^{14} ciphertext structures and 13 neutral bits, the maximum number of plaintexts required is $2^{14} \times 2^{13} \times 4 = 2^{29}$ in the worst-case scenario. In our experiments, the data complexity is often lower, as fewer structures suffice when the right structure appears earlier in the sequence.
- 2) Time complexity. By experiment, running BKS_1 once takes approximately $2^{26.27}$ times the duration of an encryption operation. In the worst case, the total time spent on the BAYESIANKEYSEARCH algorithm is equivalent to performing up to $2^{14} + 2^{14} \times 12 \times \frac{1}{2} \approx 2^{16.81}$ instances of BKS_1 , where BKS_1 runs up to 2^{14} times and BKS_2 up to $2^{14} \times 12$ times.¹⁰ Hence, the worst-case time complexity is $2^{16.81+26.27} = 2^{43.08}$ 18-round encryption operations.

7 Improved Key Recovery on Round-Reduced SIMECK32/64

In [ZLWL23], Zhang *et al.* proposed the key-recovery attacks on 16- and 17-round SIMECK32/64 using the 3-round differential combined with $\mathcal{ND}$ s. In this section, by applying the parameter optimization strategy proposed in Section 5, new parameters and improved attack complexity were obtained.

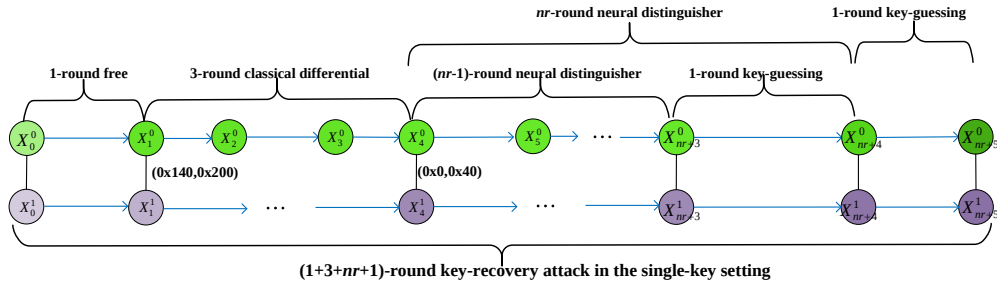


Figure 8: Components of the key-recovery attacks of round-reduced SIMECK32/64

7.1 Brief Descriptions of Key Recovery Attacks in [ZLWL23]

As shown in Figure 8, Zhang *et al.* integrated a 3-round classical differential $(0x140, 0x200) \rightarrow (0x0, 0x40)$ holding the probability $prob. \approx 2^{-7.3}$ with $(nr - 1)$ -round and nr -round $\mathcal{ND}$ s to form $(nr + 2)$ -round and $(nr + 3)$ -round hybrid distinguishers. Employing the $(nr + 2)$ -round and $(nr + 3)$ -round hybrid distinguishers, Zhang *et al.* proposed the $(nr + 5)$ -round key-recovery attacks by guessing the last two subkeys using the BAYESIANKEYSEARCH.

In the 16-round attack, 13 neutral bits, $\{[3], [4], [5], [7], [8], [9], [13], [14], [15], [18], [20], [22], [24]\}$, are used to generate the ciphertext structures. And each ciphertext structure contains 2^{13} pairs of ciphertext. The parameters for the attack are $(Cand_1 =$

¹⁰Since BKS_1 recommends 160 candidates and BKS_2 recommends 80, running BKS_1 once is approximately equal to running BKS_2 twice.

32, $Step_1 = 5, c_1 = 10, Cand_2 = 32, Step_2 = 5, c_2 = 10$), with 2^{10} ciphertext structures used to recover the last and second-to-last subkeys. The BKS_1 algorithm is executed up to $n_{it} = 2^{11}$ times, with ciphertext structures chosen based on a priority formula detailed in [Goh19, BGL⁺22]. Moreover, a practical rate of the right structures passing the cutoffs c_1 and c_2 is given using the initial parameters. Using 10^3 right structures and ($Cand_1 = 32, Step_1 = 5, c_1 = 10, Cand_2 = 32, Step_2 = 5, c_2 = 10$), $P_{BKS_1, BKS_2}^r = 0.3957$.

In the 17-round attack, 15 neutral bits, $\{[3], [4], [5], [7], [8], [9], [13], [14], [15], [18], [20], [22], [24], [30], [0, 31]\}$, are used to generate the ciphertext structures. And each ciphertext structure contains 2^{15} pairs of ciphertext. The parameters for the attack are ($Cand_1 = 32, Step_1 = 5, c_1 = 20, Cand_2 = 32, Step_2 = 5, c_2 = -120$), with 2^{10} ciphertext structures used to recover the last and second-to-last subkeys. The BKS_1 algorithm is executed up to $n_{it} = 2^{11}$ times, with ciphertext structures chosen based on a priority formula detailed in [Goh19, BGL⁺22]. A practical rate of the right structures passing the cutoffs c_1 and c_2 is given using the initial parameters. Using 10^3 right structures and ($Cand_1 = 32, Step_1 = 5, c_1 = 20, Cand_2 = 32, Step_2 = 5, c_2 = -120$), $P_{BKS_1, BKS_2}^r = 0.0836$.

7.2 Overview of Improved Attack

The idea of our improved attack is to reduce the total times of performing BAYESIANKEYSEARCH and employ fewer decryption operations. With the less computational overhead, the time complexity is improved. Then, by running the parameter optimization strategy in Section 5, we selected the appropriate values for the cutoffs c_1 and c_2 to maintain or improve the key recovery success rate. The key recovery attacks is improved in the following ways:

1. *Recovering the subkeys using the staged method.*
 - 1). In [ZLWL23], Zhang *et al.* generated the 2^{10} ciphertext structures and executed the BAYESIANKEYSEARCH on the structure chosen by the structure selection policy based on a standard exploration-exploitation technique, namely Upper Confidence Bounds. And the key recovery attack continues until either a ciphertext structure reaches the VS stage or the times of executing BKS_1 reaches a predefined limit $n_{it} = 2^{11}$. Since the maximum execution count of BKS_1 exceeds the number of structures, BKS_1 may execute more than once for certain structures. Using the configuration from [ZLWL23] to re-run the key-recovery attacks, it is found that successful attacks occur most frequently when BKS_1 is reapplied to the right structures.
 - 2). Considering that there is the higher probability that the right structure is chosen again by the structure selection policy after executing the BKS_1 once on each structure, we can first generate a small amount of structures and run the key-recovery attack on these structures. If the subkeys are guessed correctly, the attack is terminated, otherwise a batch of new structures are generated until the number of used structures reaches 2^{10} .
 - 3). In the first stage of the improved attacks, we initially generate 2^9 structures and set the maximum execution count of BKS_1 on these structures to $n_{it} = 2^9 + 2^8$. The attack terminates if the subkeys are successfully recovered using these structures. However, if the execution times of BKS_1 on the initial 2^9 structures reaches $n_{it} = 2^9 + 2^8$, we generate a new set containing 2^9 structures in the second stage and limit the number of execution times of BKS_1 to a maximum of 768 times.¹¹

¹¹We also perform key-recovery attacks using different subset sizes. Among these, the subset comprising 2^9 structures demonstrates optimal performance.

2. *Reducing the number of the candidate subkeys recommended by the BAYESIANKEY-SEARCH.* Using the configuration from [ZLWL23], there are $Cand \times Step = 160$ candidates after executing the BAYESIANKEYSEARCH once. And there are $160 \times N$ 1-round decryption operations in total after executing the BKS_1 or the BKS_2 once. Thus, an effective method to improve the time complexity is to reduce the number of the candidates.
3. *Reducing the number of the ciphertext pairs used as the input of BKS_2 .* The number of ciphertext pairs in the ciphertext structure is related to the accuracy of the combined response of the nr -round $\mathcal{ND}$, ensuring it remains above 99%. In the 16-round attacks, using 2^{13} ciphertext pairs¹², the accuracy of the combined response of the 11-round $\mathcal{ND}$ is over 99%. However, for the 10-round $\mathcal{ND}$, the accuracy of the combined response is over 99% only using 2^{11} ciphertext pairs. Thus, we can use the fewer ciphertext pairs in the BKS_2 . In the improved 17-round attack, 50% of the ciphertext pairs in each structure serve as input to BKS_2 ¹³; in the improved 16-round attack, this proportion is reduced to 25%.

Remark 2. In [ZLWL23], Zhang *et al.* made the source code available, but did not provide the trained neural distinguishers. We retrained the 10-round, 11-round and 12-round neural distinguishers of SIMECK32/64. The wrong key response profile for these neural distinguishers is shown in Appendix E. Moreover, Zhang *et al.* modified the BAYESIANKEYSEARCH algorithm by ignoring the 12-th and 13-th bit subkeys (see Algorithm 9).

7.3 Improved Single-Key Recovery Attack on 16-Round SIMECK32/64

We replace $(Cand_1 = 32, Step_1 = 5, Cand_2 = 32, Step_2 = 5)$ with $(Cand_1 = 16, Step_1 = 5, Cand_2 = 16, Step_2 = 5)$. Running our parameter optimization strategy with $p_{>c_1}^r = 0.6$, the values of (c_1, c_2) are determined to be $(0.5656, -135)$.

The high-quality difference set used in Algorithm 5 is Table 8.

Table 8: The high-quality difference set. These high-quality differences appear frequently in the output of Algorithm 2 using the 2^{12} right structures. $\text{Diff}_{\text{HW}(\Delta) \leq 2}$ refers to the set holding the difference with the hamming weight lower than 2.

Diff ₀	{0x0}	Diff ₁	{0x80}	Diff ₂	{0x100}	Diff ₃	{0x280}
Diff ₄	{0x200}	Diff ₅	{0x4}	Diff ₆	{0x300}	Diff ₇	{0x180}
Diff ₈	{0x1000}	Diff ₉	{0x2300}	Diff ₁₀	{0x380}	Diff ₁₁	{0x2000}
Diff ₁₂	{0x84}	Diff ₁₃	{0x40}	Diff ₁₄	{0x104}	Diff ₁₅	{0x2104}
Diff ₁₆	{0xc0}	Diff ₁₇	{0x2200}	Diff ₁₈	{0x1080}	Diff ₁₉	{0x204}
Diff ₂₀	$\text{Diff}_{\text{HW}(\Delta) \leq 2} \setminus \bigcup_{i=0}^{19} \text{Diff}_i$						

Comparative analysis of computational cost per wrong structure. In [ZLWL23], using $(Cand_1 = 32, Step_1 = 5, c_1 = 10.0, Cand_2 = 32, Step_2 = 5)$, there is $p_{>c_1}^w = 2.52 \times 10^{-3}$, where $p_{>c_1}^w$ is the probability that there is at least one candidate with a score higher than c_1 when executing BKS_1 on one wrong structure. And for the wrong structure holding the candidate with a score higher than c_1 , there are 2.27 candidates with scores higher than c_1 on average. Thus, the average computational time on one wrong structure is

¹²In [ZLWL23], each sample contains 2^3 ciphertext pairs. And when using 2^{10} samples holding the same distribution, the accuracy of the combined response of the 11-round $\mathcal{ND}$ is over 99%.

¹³When only 25% of the ciphertext pairs per structure serve as input to BKS_2 , the quality of its recommendations significantly deteriorates.

$T = t_1 + p_{>c_1}^w \times 2.27 \times t_2 = t_1 + 5.72 \times 10^{-3} \times t_2$, where the t_1 is the time of running the BKS_1 once and the t_2 is the time of running the BKS_2 once.

Using the new parameters selected by our parameter optimization strategy, there is $p_{>c_1}^w = 7.23 \times 10^{-3}$. And there are 1.14 candidates with scores higher than c_1 on average. Thus, the average computational time on one wrong structure is $\bar{T} = \bar{t}_1 + p_{>c_1}^w \times 1.14 \times \bar{t}_2 = \bar{t}_1 + 8.24 \times 10^{-3} \times \bar{t}_2$, where the $\bar{t}_1$ is the time of running the BKS_1 once and the $\bar{t}_2$ is the time of running the BKS_2 once.

Due to that there are fewer candidates in BKS_1 using the new parameters, there is $\bar{t}_1 < t_1$. For BKS_2 , there are 160×2^{13} 1-round decryption operations using the parameters from [ZLWL23]. Using the new parameters, there are 80×2^{11} 1-round decryption operations. Thus, there is $\bar{t}_2 < \frac{1}{8} \times t_2$. Therefore, $\bar{T} < T$.

Due to that there are only a few right structures, the attack runtime is dominated by executing the BAYESIANKEYSEARCH on wrong structures. Consequently, the new parameters yield reduced computational costs.

Success rate. We count the key-recovery attack as successful if the right ciphertext structure is used as the input of VS . Under these conditions, the attacks are successful in 120 out of 120 trails.

Data complexity. Among the 120 trails, there are 115 trails that the subkeys are guessed correctly only using the first 2^9 structures. The data complexity is 2^{24} plaintexts in the worst case. The actual data complexity is $2^{22.18}$ plaintexts.

Time complexity. The average running time of the experiment is 1149.06 s. Our device can perform $2^{27.13}$ 1-round decryption per second. Thus, the time complexity is $2^{27.13} \times 1149.06 = 2^{37.29}$.

7.4 Improved Single-Key Recovery Attack on 17-Round SIMECK32/64

We replace $(Cand_1 = 32, Step_1 = 5, Cand_2 = 32, Step_2 = 5)$ with $(Cand_1 = 32, Step_1 = 5, Cand_2 = 16, Step_2 = 5)$. Running our parameter optimization strategy with the different values of $p_{>c_1}^r$, the values of (c_1, c_2) are shown in Table 9.

Table 9: Recommended cutoffs with $(Cand_1, Step_1, Cand_2, Step_2) = (32, 5, 16, 5)$ and $n_{it} = 2^{10} + 2^9$.

$p_{>c_1}^r$	c_1	c_2	P_{BKS_1, BKS_2}^r
0.2	18.0287	-21	0.0815 with S_{256}
0.3	13.9906	-15	0.0916 with S_{256}
0.4	10.7688	-11	0.10057 with S_{256}

Using the parameters $(Cand_1, Step_1, c_1, Cand_2, Step_2, c_2) = (32, 5, 13.9906, 16, 5, -15)$, we perform 50 key recovery attacks.

Success rate. We count the key-recovery attack as successful if the right ciphertext structure is used as the input of VS . Under these conditions, the attacks are successful in 17 out of 50 trails. Thus, the success rate is 34%.

Data complexity. The data complexity is 2^{24} plaintexts in the worst case.

Time complexity. The average running time of the experiment is 8098.104 s. Our device can perform $2^{27.13}$ 1-round decryption per second. In [ZLWL23], the time complexity calculation formula is $num_{enc} \times rt \times \log_{1-sr} 0.01$, where num_{enc} is the number of time of performing 1-round decryption per round, rt is the average running time, sr is the success rate. Thus, the time complexity using the new parameters is $2^{43.58}$, which is lower than using the initial parameters in [ZLWL23].

8 Runtime of the Parameter Optimization Strategy

For the related-key attack on 18-round SIMON32/64, the run time is related to the data-gathering phase as follows.

1. **Stage1: Calculating the value of c_1 .** In this stage, Algorithm 2 is executed using the right ciphertext structures as the input. In Algorithm 2, we run the BKS_1 on 2^{14} right ciphertext structures for each pair of $(Cand_1, Step_1)$ and save the differences between the candidates and the last subkeys and their scores.
2. **Stage2: Calculating the value of $p_{>c_1}^w$.** In this stage, Algorithm 2 is executed with the wrong ciphertext structures as the input. In Algorithm 2, we run the BKS_1 on 2^{14} wrong ciphertext structures for each pair of $(Cand_1, Step_1)$ and save the differences between the candidates and the last subkeys and their scores.
3. **Stage3: Generating the data for the distribution fitting.** In this stage, Algorithm 3 is executed with the wrong ciphertext structures as the input. In Algorithm 3, we run the BKS_2 on 2^{13} wrong ciphertext structures for each pair of $(Cand_2, Step_2)$ and save the maximum value among all scores in each output of performing BKS_2 .
4. **Stage4: Calculating the value of P_{BKS_1, BKS_2}^r .** In Algorithm 5, Algorithm 4 is executed m times on 2^{13} right structures, where the m is the number of subsets of high-quality differences. During the parameter optimization strategy of the 18-round related-key attack, for each pair $(Cand_2, Step_2)$, there are 5 high-quality difference set used to calculate P_{BKS_1, BKS_2}^r , S_{16} , S_{32} , S_{64} , S_{128} and S_{256} . Thus, we run the BKS_2 on 5×2^{13} right ciphertext structures for each pair $(Cand_2, Step_2)$.

For each $(Cand_1, Step_1, Cand_2, Step_2)$, the runtime is less than the total time of performing BKS_1 2^{15} times (2^{14} times in the **Stage 1** and 2^{14} times in the **Stage 2**) and performing BKS_2 6×2^{13} times (2^{13} times in the **Stage 3** and 5×2^{13} times in the **Stage 4**).

With the values of $(Cand_1, Step_1, Cand_2, Step_2)$ shown in Table 4, there holds $Cand_2 \times Step_2 \leq Cand_1 \times Step_1$ for most values of $(Cand_1, Step_1)$, which means that the time of running BKS_2 one time is lower than that of running BKS_1 one time.

Thus, the total runtime of executing our parameter optimization strategy is less than the time of performing BKS_1 5×2^{14} times.

For the worst case of performing the key-recovery attack, there are performing BKS_1 2^{14} times and performing the BKS_2 $2^{14} \times n_{>c_1}$ times when using 2^{14} ciphertext structures.

Based on the above analysis, the runtime of performing the parameter optimization strategy one time is less than performing worst-case key-recovery attacks five times.

9 Conclusion and Future Work

This paper investigates the relations between the parameters of BAYESIANKEYSEARCH, the cutoffs used, the success rate, and the attack complexity in machine learning-aided

key-recovery attacks. Based on observations and analysis, we proposed a coordinated strategy for selecting appropriate parameters and cutoffs for both BAYESIANKEYSEARCH stages to optimize both the effectiveness and efficiency of the attack. Applying this strategy, we enhanced the key-recovery attack on 16-round SIMON32/64, 16-round and 17-round SIMECK32/64 in the single-key setting and proposed new attacks on 18-round SIMON32/64 in the related-key setting, achieving experimental results that closely match theoretical success rates. Additionally, our improved related-key neural distinguishers show that incorporating more observable features enhances accuracy.

Despite these advancements, our parameter selection strategy involves precomputation and local search, leaving open questions about the existence of globally optimal parameters. Furthermore, the relationship between the performance of BAYESIANKEYSEARCH and the number of recommended subkey candidates per iteration remains unclear.

Acknowledgments

The authors would like to thank anonymous reviewers and shepherd for their insightful and helpful comments which helped us improve the manuscript significantly. This research is supported in part by the National Natural Science Foundation of China (No. 62372262) and the National Key R&D Program of China (No. 2023YFA1011200), and Zhongguancun Laboratory.

References

- [AAA⁺15] Mohamed Ahmed Abdelraheem, Javad Alizadeh, Hoda A. AlKhzaimi, Mohammad Reza Aref, Nasour Bagheri, and Praveen Gauravaram. Improved linear cryptanalysis of reduced-round SIMON-32 and SIMON-48. In Alex Biryukov and Vipul Goyal, editors, *INDOCRYPT 2015*, volume 9462 of *LNCS*, pages 153–179. Springer, Heidelberg, December 2015.
- [ALLW15] Farzaneh Abed, Eik List, Stefan Lucks, and Jakob Wenzel. Differential cryptanalysis of round-reduced Simon and Speck. In Carlos Cid and Christian Rechberger, editors, *FSE 2014*, volume 8540 of *LNCS*, pages 525–545. Springer, Heidelberg, March 2015.
- [Bag15] Nasour Bagheri. Linear cryptanalysis of reduced-round SIMECK variants. In Alex Biryukov and Vipul Goyal, editors, *INDOCRYPT 2015*, volume 9462 of *LNCS*, pages 140–152. Springer, Heidelberg, December 2015.
- [BC04] Eli Biham and Rafi Chen. Near-collisions of SHA-0. In Matthew Franklin, editor, *CRYPTO 2004*, volume 3152 of *LNCS*, pages 290–305. Springer, Heidelberg, August 2004.
- [BGHR23] Emanuele Bellini, David Gérardt, Anna Hambitzer, and Matteo Rossi. A cipher-agnostic neural training pipeline with automated finding of good input differences. *IACR Trans. Symm. Cryptol.*, 2023(3):184–212, 2023.
- [BGL⁺21] Zhenzhen Bao, Jian Guo, Meicheng Liu, Li Ma, and Yi Tu. Conditional differential-neural cryptanalysis. Cryptology ePrint Archive, Report 2021/719, 2021. <https://eprint.iacr.org/2021/719>.
- [BGL⁺22] Zhenzhen Bao, Jian Guo, Meicheng Liu, Li Ma, and Yi Tu. Enhancing differential-neural cryptanalysis. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part I*, volume 13791 of *LNCS*, pages 318–347. Springer, Heidelberg, December 2022.

- [BGPT21] Adrien Benamira, David G rault, Thomas Peyrin, and Quan Quan Tan. A deeper look at machine learning-based cryptanalysis. In Anne Canteaut and Fran ois-Xavier Standaert, editors, *EUROCRYPT 2021, Part I*, volume 12696 of *LNCS*, pages 805–835. Springer, Heidelberg, October 2021.
- [BLYZ23] Zhenzhen Bao, Jinyu Lu, Yiran Yao, and Liu Zhang. More insight on deep learning-aided cryptanalysis. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part III*, volume 14440 of *LNCS*, pages 436–467. Springer, Heidelberg, December 2023.
- [BSS⁺13] Ray Beaulieu, Douglas Shors, Jason Smith, Stefan Treatman-Clark, Bryan Weeks, and Louis Wingers. The SIMON and SPECK families of lightweight block ciphers. Cryptology ePrint Archive, Report 2013/404, 2013. <https://eprint.iacr.org/2013/404>.
- [CSYY23] Yi Chen, Yantian Shen, Hongbo Yu, and Sitong Yuan. A new neural distinguisher considering features derived from multiple ciphertext pairs. *The Computer Journal.*, 66(6):1419–1433, 2023.
- [DZW25] Weiqing Deng, Jianing Zhang, and Haoyang Wang. Improved differential meet-in-the-middle cryptanalysis on SIMON and piccolo. In Willy Susilo and Josef Pieprzyk, editors, *ACISP 25*, volume 15658 of *LNCS*, pages 78–97. Springer, Heidelberg, July 2025.
- [GHHP24] David G rault, Anna Hambitzer, Moritz Huppert, and Stjepan Picek. Sok: 5 years of neural differential cryptanalysis. Cryptology ePrint Archive, Report 2024/1300, 2024. <https://eprint.iacr.org/2024/1300>.
- [GLN22] Aron Gohr, Gregor Leander, and Patrick Neumann. An assessment of differential-neural distinguishers. Cryptology ePrint Archive, Report 2022/1521, 2022. <https://eprint.iacr.org/2022/1521>.
- [Goh19] Aron Gohr. Improving attacks on round-reduced Speck32/64 using deep learning. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 150–179. Springer, Heidelberg, August 2019.
- [HWZ18] Mingjiang Huang, Liming Wang, and Yan Zhang. Improved automatic search algorithm for differential and linear cryptanalysis on SIMECK and the applications. In David Naccache, Shouhuai Xu, Sihang Qing, Pierangela Samarati, Gregory Blanc, Rongxing Lu, Zonghua Zhang, and Ahmed Meddahi, editors, *Information and Communications Security - 20th International Conference, ICICS 2018, Lille, France, October 29-31, 2018, Proceedings*, volume 11149 of *Lecture Notes in Computer Science*, pages 664–681. Springer, 2018.
- [KPH⁺19] Jaehun Kim, Stjepan Picek, Annelie Heuser, Shivam Bhasin, and Alan Hanjalic. Make some noise: Unleashing the power of convolutional neural networks for profiled side-channel analysis. *IACR TCHES*, 2019(3):148–179, 2019. <https://tches.iacr.org/index.php/TCHES/article/view/8292>.
- [KR15] Stefan K lbl and Arnab Roy. A brief comparison of Simon and Simeck. Cryptology ePrint Archive, Report 2015/706, 2015. <https://eprint.iacr.org/2015/706>.
- [LLS⁺24] Jinyu Lu, Guoqiang Liu, Bing Sun, Chao Li, and Li Liu. Improved (related-key) differential-based neural distinguishers for SIMON and SIMECK block ciphers. *The Computer Journal.*, 67(2):537–547, 2024.

- [LPS21] Gaëtan Leurent, Clara Pernot, and André Schrottenloher. Clustering effect in simon and simeck. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part I*, volume 13090 of *LNCS*, pages 272–302. Springer, Heidelberg, December 2021.
- [LTZ22] Lijun Lyu, Yi Tu, and Yingjie Zhang. Deep learning assisted key recovery attack for round-reduced simeck32/64. In Willy Susilo, Xiaofeng Chen, Fuchun Guo, Yudi Zhang, and Rolly Intan, editors, *Information Security - 25th International Conference, ISC 2022, Bali, Indonesia, December 18-22, 2022, Proceedings*, volume 13640 of *Lecture Notes in Computer Science*, pages 443–463. Springer, 2022.
- [PPM⁺23] Stjepan Picek, Guilherme Perin, Luca Mariot, Lichao Wu, and Lejla Batina. Sok: Deep learning-based physical side-channel analysis. *ACM Comput. Surv.*, 55(11):227:1–227:35, 2023.
- [PSK⁺18] Stjepan Picek, Ioannis Petros Samiotis, Jaehun Kim, Annelie Heuser, Shivam Bhasin, and Axel Legay. On the performance of convolutional neural networks for side-channel analysis. In Anupam Chattopadhyay, Chester Rebeiro, and Yuval Yarom, editors, *Security, Privacy, and Applied Cryptography Engineering - 8th International Conference, SPACE 2018, Kanpur, India, December 15-19, 2018, Proceedings*, volume 11348 of *Lecture Notes in Computer Science*, pages 157–176. Springer, 2018.
- [QHS16] Kexin Qiao, Lei Hu, and Siwei Sun. Differential analysis on simeck and SIMON with dynamic key-guessing techniques. In Olivier Camp, Steven Furnell, and Paolo Mori, editors, *Information Systems Security and Privacy - Second International Conference, ICISSP 2016, Rome, Italy, February 19-21, 2016, Revised Selected Papers*, volume 691 of *Communications in Computer and Information Science*, pages 64–85. Springer, 2016.
- [Riv93] Ronald L. Rivest. Cryptography and machine learning (invited lecture). In Hideki Imai, Ronald L. Rivest, and Tsutomu Matsumoto, editors, *ASIACRYPT'91*, volume 739 of *LNCS*, pages 427–439. Springer, Heidelberg, November 1993.
- [SLJ⁺15] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott E. Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2015, Boston, MA, USA, June 7-12, 2015*, pages 1–9. IEEE Computer Society, 2015.
- [WLV⁺14] Qingju Wang, Zhiqiang Liu, Kerem Varici, Yu Sasaki, Vincent Rijmen, and Yosuke Todo. Cryptanalysis of reduced-round SIMON32 and SIMON48. In Willi Meier and Debdeep Mukhopadhyay, editors, *INDOCRYPT 2014*, volume 8885 of *LNCS*, pages 143–160. Springer, Heidelberg, December 2014.
- [WWJZ18] Ning Wang, Xiaoyun Wang, Keting Jia, and Jingyuan Zhao. Differential attacks on reduced SIMON versions with dynamic key-guessing techniques. *Sci. China Inf. Sci.*, 61(9):098103:1–098103:3, 2018.
- [YLL⁺22] Geng Yuan, Yanyu Li, Sheng Li, Zhenglun Kong, Sergey Tulyakov, Xulong Tang, Yanzhi Wang, and Jian Ren. Layer freezing & data sieving: Missing pieces of a generic framework for sparse training. In Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information*

Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022, 2022.

- [YZS⁺15] Gangqiang Yang, Bo Zhu, Valentin Suder, Mark D. Aagaard, and Guang Gong. The simeck family of lightweight block ciphers. In Tim Güneysu and Helena Handschuh, editors, *CHES 2015*, volume 9293 of *LNCS*, pages 307–329. Springer, Heidelberg, September 2015.
- [ZLQ⁺21] Yingjie Zhang, Lijun Lyu, Kexin Qiao, Zhiyu Zhang, Siwei Sun, and Lei Hu. Automatic key recovery of feistel ciphers: Application to SIMON and SIMECK. In Robert H. Deng, Feng Bao, Guilin Wang, Jian Shen, Mark Ryan, Weizhi Meng, and Ding Wang, editors, *Information Security Practice and Experience: 16th International Conference, ISPEC 2021, Nanjing, China, December 17-19, 2021, Proceedings*, volume 13107 of *Lecture Notes in Computer Science*, pages 147–167. Springer, 2021.
- [ZLWL23] Liu Zhang, Jinyu Lu, Zilong Wang, and Chao Li. Improved differential-neural cryptanalysis for round-reduced SIMECK32/64. *Frontiers Comput. Sci.*, 17(6):176817, 2023.
- [ZWW22] Liu Zhang, Zilong Wang, and Boyang Wang. Improving differential-neural cryptanalysis with inception blocks. Cryptology ePrint Archive, Report 2022/183, 2022. <https://eprint.iacr.org/2022/183>.
- [ZZW24] Weixi Zheng, Liu Zhang, and Zilong Wang. Theoretical explanation and improvement of deep learning-aided cryptanalysis. Cryptology ePrint Archive, Report 2024/322, 2024. <https://eprint.iacr.org/2024/322>.

A Brief Descriptions of SIMON32/64 and SIMECK32/64

The SIMON32/64 and SIMECK32/64 are the small members of SIMON [BSS⁺13] and SIMECK [YZS⁺15] family, using a 16-bit subkey and two 16-bit words in the round function with a 64-bit master key. The round functions use combinations of bitwise rotation, bitwise XOR, and bitwise AND. The encryption algorithms of SIMON32/64 and SIMECK32/64 are listed in Algorithm 7 and Algorithm 8. The key schedule of SIMON32/64 uses the linear function only containing bitwise rotation and XOR. And the key schedule of SIMECK32/64 uses the non-linear function containing bitwise rotation, XOR and AND.

Algorithm 7 Encryption of SIMON32/64

Input: $P, \{k_0, k_1, \dots, k_{31}\}$

Output: C

```

1:  $(L_0, R_0) \leftarrow P$ 
2: for  $i \in \{0, \dots, 31\}$  do
3:    $L_{i+1} \leftarrow R_i \oplus (L_i \lll^{11} \odot L_i \lll^{8}) \oplus L_i \lll^{2} \oplus k_i$ 
4:    $R_{i+1} \leftarrow L_i$ 
5: end for
6:  $C \leftarrow L_{32} || R_{32}$ 
7: return  $C$ 

```

Algorithm 8 Encryption of SIMECK32/64

Input: $P, \{k_0, k_1, \dots, k_{31}\}$

Output: C

```

1:  $(L_0, R_0) \leftarrow P$ 
2: for  $i \in \{0, \dots, 31\}$  do
3:    $L_{i+1} \leftarrow R_i \oplus (L_i \odot L_i \lll^{5}) \oplus L_i \lll^{1} \oplus k_i$ 
4:    $R_{i+1} \leftarrow L_i$ 
5: end for
6:  $C \leftarrow L_{32} || R_{32}$ 
7: return  $C$ 

```

In Algorithm 7 and Algorithm 8, bitwise XOR will in the sequel be denoted by $\oplus$, bitwise AND by $\odot$, bitwise left (resp. right) rotation by s positions by $\lll s$ (resp. $\ggg s$). Denote by (L_i, R_i) the left and right branches of a state after the encryption of i rounds. Denote by $L_i || R_i$ the concatenation of L_i and R_i . Denote by $L_i[j]$ (resp. $R_i[j]$) the j -th bit of L_i (resp. R_i) counted starting from 0, where $L_i[0]$ (resp. $R_i[0]$) is the LSB (Least Significant Bit) of L_i (resp. R_i), i.e., $L_i = L_i[n-1] || L_i[n-2] || \dots || L_i[0]$ and $R_i = R_i[n-1] || R_i[n-2] || \dots || R_i[0]$.

B Brief Descriptions of the Wrong Key Response Profile

Given the neural distinguisher of the r -round target cipher holding the n -bit subkey, its corresponding wrong key response profile is calculated as follows:

1. Randomly generate N plaintexts, i.e., P .
2. Generate plaintext pairs, i.e., (P, P') , with the input difference of the neural distinguisher.
3. Query for the ciphertexts under $(r+1)$ -round target cipher using N master keys, obtaining N ciphertext pairs, denoted by $\mathcal{C}$.
4. For d from 0 to $2^n - 1$:
 - 1). Decrypt the ciphertext pair in $\mathcal{C}$ by one round using the subkey $rk \oplus d$, where the rk is the real subkey.
 - 2). Use these ciphertext pairs decrypted one round as the input of the r -round neural distinguisher, and obtain N returned values.
 - 3). Calculate the mean and standard deviation of these N returned values, denoted by μ_d and σ_d , respectively.

The $\{(\mu_d, \sigma_d) | d \in \mathbb{F}_2^n\}$ is regarded as the wrong key response profile of the r -round neural distinguisher. Given the difference d , its mean response is the value of μ_d .

C The Features of Candidates Passing c_1

To research what kind of candidates is more likely to pass c_1 , [Algorithm 2](#) is performed using 2^{12} right ciphertext structures. For the output of [Algorithm 2](#), the difference between the candidate and the real last subkey is saved when its corresponding score is higher than c_1 . The frequency distributions of these saved differences are shown in [Figure 9](#).

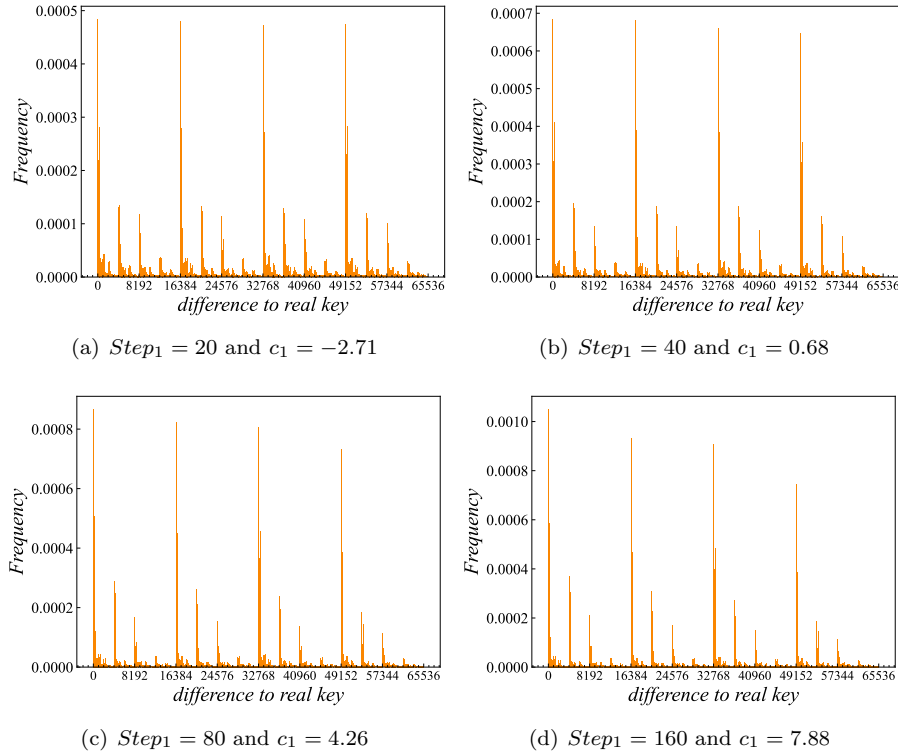


Figure 9: The frequency distribution of the difference passing c_1 with $p_{>c_1}^r = 0.9691$, $Cand_1 = 4$ and $Step_1 \in \{20, 40, 80, 160\}$

An intriguing phenomenon shown in [Figure 9](#) is that the probability of the candidates passing c_1 is related to the location of non-zero bit of the difference between candidate and the real last subkey. Similar situations occur with the wrong key response profile, which is shown in [Figure 10](#). Thus, it is logical to suppose that the probability of the candidates passing c_1 is positively related to μ_δ in the wrong key response profile, where the difference between the candidate and the real subkey is δ . It is more likely to pass c_1 for the candidate if the difference between the candidate and the real subkey has higher mean response in wrong key response profile.

D Comparison with [\[BGL⁺22\]](#)

D.1 Tuning the Parameters of BKS_1

In [\[BGL⁺22\]](#), Bao *et al.* choose the value of c_1 by the distribution of the maximum score, shown in [Figure 11](#). With $p_{>c_1}^r = 0.9691$, the value of c_1 is shown in [Table 10](#).

Although there is the slight gap in the value of c_1 using two methods, the two value will be same if enough data is used in the two methods. This is due to that they're essentially reflecting the same distribution, only the different representations on $p_{>c_1}^r = 0.9691$.

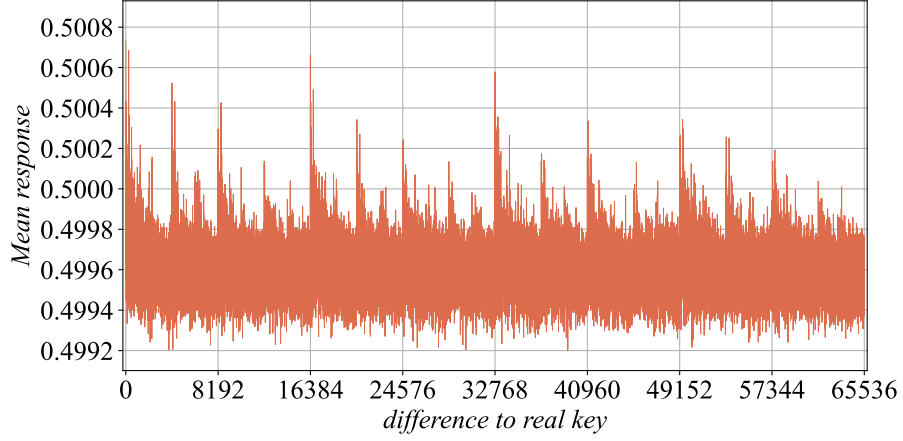


Figure 10: Wrong key response profile of μ_δ for $\mathcal{RK}\text{-}\mathcal{ND}^{\text{SIMON}_{13R}}$

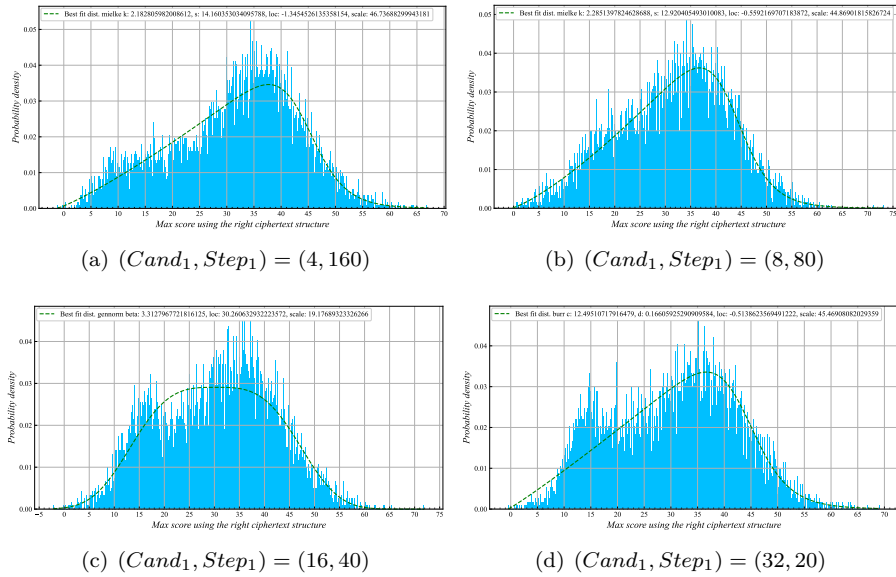


Figure 11: The distributions of the maximum score when performing BKS_1 on right ciphertext structures, with $Cand_1 \times Step_1 = 640$

Table 10: The recommended values of c_1 using the different methods.

$(Cand_1, Step_1)$	c_1 using Bao <i>et al.</i> 'method	c_1 using Algorithm 2
(4, 160)	8.16	7.88
(8, 80)	9.24	10.21
(16, 40)	9.87	10.47
(32, 20)	7.99	10.26

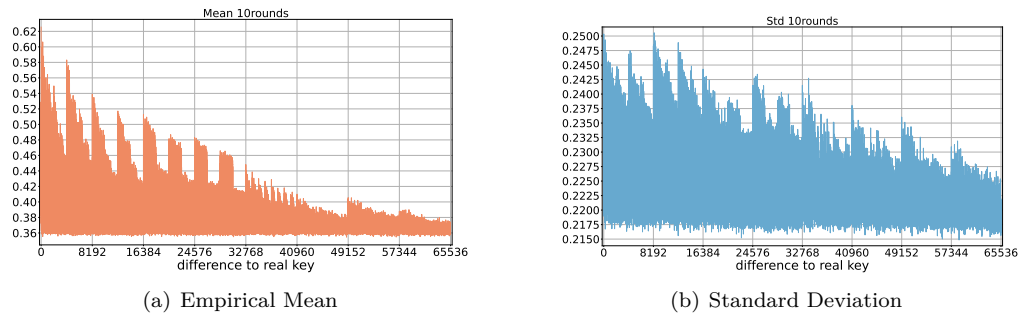
D.2 Tuning the Parameters of BKS_2

Different from selecting a value that is just large enough to be uncovered for c_2 in [BGL⁺22], we try to find the smallest possible value for c_2 , whose aim is to increase the shot percentage of the right structure.

Performing actually the key-recovery attacks, with large value of c_2 , it is found that there are a lot of right structures discarded by c_2 in the first time of facing the BKS_1 and BKS_2 and most of them are picked up again using the Upper Confidence Bounds. This kind of phenomenon is something we do not want to see. And this raises the new question about how to determine the number of loop to make sure that the right structures discarded by c_2 can be picked up again, which is fuzzy. Based on this, we want to increase the probability of the right structure passing c_2 by choosing the smallest possible value for c_2 .

On the other hand, as we mentioned in this paper, that the parameters of BAYESIANKEY-SEARCH are important for the time complexity and the success rate, we need to find quickly the best choice to perform the key-recovery attack. Without the clear description of c_2 , the success rate will only be obtained by performing actually a lot of attacks. However, it can need huge time to perform the attack one time, which makes it difficult to test multiple pairs of parameters (For the related-key attack on 18-round SIMON32/64 shown in this paper, it needs more than 6 hours given 2^{14} structures to perform the attack one time.).

E Wrong Key Response Profile and BayesianKeySearch Algorithm For Simeck32/64

**Figure 12:** Wrong key response profile for 10-round SIMECK32/64

Algorithm 9 BAYESIANKEYSEARCH Algorithm For SIMECK32/64 [ZLWL23]

Input: Ciphertext Structure: $C_{structure} := \{C_0, C_1, \dots, C_{n_b-1}\}$
 Neural Distinguisher: $\mathcal{ND}$
 Wrong Key Response Profile ($WKR\mathcal{P}$): μ and σ

Output: The list L of tuples of recommended keys and their scores

- 1: $CK := \{k_0, k_1, \dots, k_{Cand-1}\} \leftarrow$ randomly choose $Cand$ values without replacement
- 2: $CK = CK \& 0xCFFF$
- 3: $L \leftarrow \{\}$
- 4: $Loop \leftarrow 0$
- 5: **while** $Loop < Step$ **do**
- 6: $Loop \leftarrow Loop + 1$
- 7: **for** $i \in \{0, 1, \dots, Cand - 1\}$ **do**
- 8: **for** $j \in \{0, 1, \dots, n_b - 1\}$ **do**
- 9: $\overline{C}_{i,j} \leftarrow Decrypt_{1r}(C_j, k_i)$
- 10: $v_{i,j} \leftarrow \mathcal{ND}(\overline{C}_{i,j})$
- 11: $s_{i,j} \leftarrow \log_2(v_{i,j}/(1 - v_{i,j}))$
- 12: **end for**
- 13: $s_{k_i} = \sum_{j=0}^{n_b-1} s_{i,j}$
- 14: $L \leftarrow L \cup (k_i, s_{k_i})$
- 15: $m_{k_i} = \sum_{j=0}^{n_b-1} v_{i,j} \Big/ n_b$
- 16: **end for**
- 17: **for** $k \in \{0, 1, \dots, 2^{16} - 1\}$ **do**
- 18: $\lambda_k = \sum_{i=0}^{Cand-1} (m_{k_i} - \mu_{k_i \oplus k})^2 \Big/ \sigma_{k_i \oplus k}^2$
- 19: **end for**
- 20: $CK \leftarrow \text{argsort}_k(\lambda)[0 : Cand - 1]$ /* Pick $Cand$ new candidates with the $Cand$ smallest score λ_k . */
- 21: $r := \{r_0, r_1, \dots, r_{Cand-1}\} \leftarrow$ randomly choose $Cand$ values from $\{0, 1, 2, 3\}$
- 22: $r = r \ll 12$
- 23: $CK = CK \oplus r$
- 24: **end while**
- 25: **Return** L

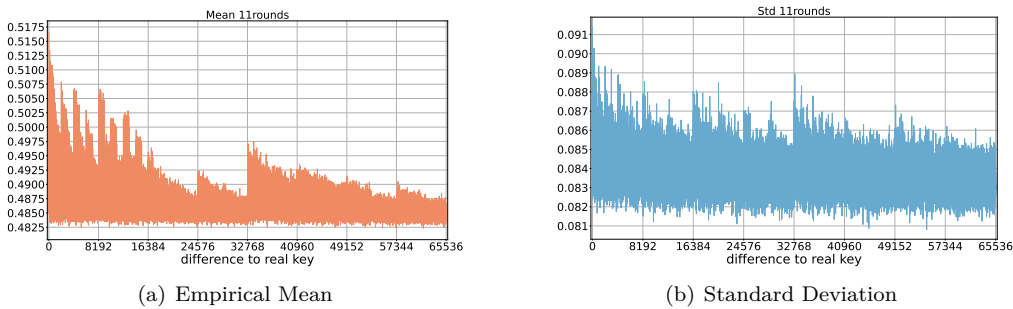


Figure 13: Wrong key response profile for 11-round SIMECK32/64

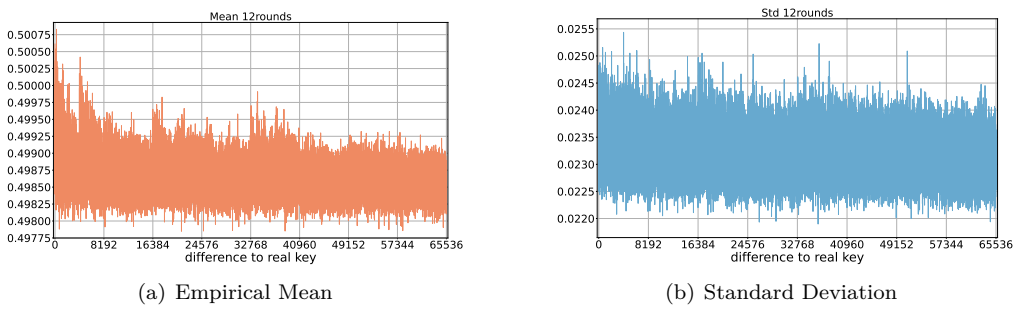


Figure 14: Wrong key response profile for 12-round SIMECK32/64